Multiprecision vs. Fixed-Width Digit Generation for JSON Canonicalization: An Empirical Study

Mark Lenhardt

Lattice Substrate

Abstract

JSON Canonicalization Scheme (RFC 8785) requires deterministic floating-point formatting that matches ECMA-262 `Number::toString`. We present the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a specification-constrained canonicalization pipeline, implemented in both Go and Rust to separate algorithmic effects from language/runtime effects. Our methodology gates performance measurement on conformance: only implementations passing 286,362 oracle vectors (generated from the V8 JavaScript engine) and 308 structural conformance cases (77 per implementation × 4 implementations) proceed to benchmarking. Using permutation tests, BCa bootstrap confidence intervals, and Benjamini–Hochberg FDR correction across 15 (CLI) and 10 (API) repeated measurements, we find that Schubfach is 1.3–1.7× faster than Burger–Dybvig on number-dense workloads in Go API benchmarks due to zero heap allocation in the digit-generation path; the advantage replicates at 1.7–2.8× in Rust, confirming it is algorithmic rather than language-specific (excluding one extreme outlier, `numeric-boundary` at 9.2× in Rust, discussed in §8). In end-to-end CLI measurement, 11 of 28 within-Go comparisons reach BH-corrected significance (1.1–1.3× speedup); the remainder are dominated by process startup overhead. On string-dominant and deeply-nested inputs the difference narrows to <5%, and on several API-level non-number workloads (e.g., `control-escapes`, `rfc-key-sorting`) Burger–Dybvig is marginally faster (≤7%). All four implementations produce byte-identical output; Go binaries are additionally verified under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). We release the benchmark infrastructure, oracle vectors, and all raw data as a reproducibility artifact.

Keywords: JSON canonicalization, floating-point formatting, Schubfach, Burger–Dybvig, RFC 8785, ECMA-262, benchmark methodology, cross-language validation

1 Introduction

The JSON Canonicalization Scheme (RFC 8785) [12] standardizes a deterministic serialization of JSON values for use in digital signatures, content-addressable storage, and distributed consensus protocols. A critical requirement is that floating-point numbers must

be formatted identically to the output of ECMA-262 `Number::toString` [5], which specifies shortest-representation formatting under roundTiesToEven semantics.

Two families of algorithms can satisfy this requirement: multiprecision arithmetic approaches exemplified by Burger–Dybvig [3], and fixed-width integer approaches exemplified by Schubfach [7]. While these algorithms are individually well-studied, no prior work has empirically compared them *inside a specification-constrained canonicalization pipeline* where conformance is a hard prerequisite and the formatting stage is only one component of the total cost.

This paper makes three contributions:

1. The first empirical comparison of multiprecision vs. fixed-width digit generation under ECMA-262/RFC 8785 constraints, showing that Schubfach achieves 1.3–1.7× speedup on number-dense workloads in API benchmarks; in end-to-end CLI measurement, the BH-significant comparisons show 1.1–1.3× speedup. String-dominant and deeply-nested inputs show <5% difference.

2. A benchmark methodology that gates performance measurement on conformance verification: implementations must pass 286,362 oracle vectors and 308 structural conformance cases before timing data is collected. We document noise floors and minimum observable effect sizes for all comparisons.

3. Cross-compilation determinism verification showing byte-identical output under Go cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host), with all evidence released as a reproducibility artifact.

2 Background

2.1 RFC 8785: JSON Canonicalization Scheme

RFC 8785 [12] defines a deterministic JSON serialization by specifying: (1) lexicographic key ordering using UTF-16 code unit comparison, (2) no insignificant whitespace, and (3) floating-point formatting matching ECMA-262 `Number::toString`.

The floating-point requirement is the most technically demanding. It requires producing the *shortest decimal representation* that round-trips to the original IEEE 754 binary64 value under roundTiesToEven, then applying the ECMA-262 formatting policy (exponential notation thresholds, sign handling, integer elision rules).

2.2 ECMA-262 Number Formatting

ECMA-262 [5] §6.1.6.1.20 specifies `Number::toString` as a two-stage process (formerly §7.1.12.1 in older editions):

1. **Digit generation**: find the shortest decimal significand d and exponent n such that $d \times 10^{n-k} = x$ (where k is the number of digits in d) and the value round-trips.

2. **Formatting**: apply branch logic based on the relationship between k (digit count) and n (decimal exponent position) to produce the final string. Key thresholds: $n \geq 1 \wedge n \leq 21$ produces fixed notation; $n > 21$ or $n \leq 0$ with $|n| < \mathit{certainbounds}$ produces exponential notation.

2.3 IEEE 754 and Shortest Representation

IEEE 754-2019 [8] binary64 has a 52-bit significand, 11-bit exponent, and 1 sign bit. The *shortest representation* property requires finding the decimal string with fewest digits that uniquely identifies the original binary64 value among all representable values. This is the core algorithmic challenge addressed by both Burger–Dybvig and Schubfach.

3 Related Work

Extensive prior work has optimized float-to-string conversion in isolation. This paper is the first to evaluate these algorithms inside a specification-constrained canonicalization pipeline where conformance is a binary gate. We summarize the most relevant prior work, organized by the problem dimension each addresses.

3.1 Multiprecision Foundations

Burger and Dybvig [3] establish the theoretical framework for shortest-representation printing using arbitrary-precision rational arithmetic. Their algorithm is our multiprecision baseline. Gay's `dtoa.c` [6] provides the foundational practical implementation and remains widely used in language runtimes as a reference or fallback. Both works evaluate algorithms in isolation and do not consider performance inside a pipeline where formatting policy is externally constrained.

3.2 Fixed-Width Approaches

Loitsch [11] demonstrated that integer-only arithmetic suffices for the common case with Grisu. However, Grisu3 falls back to arbitrary precision for $\sim$0.5% of inputs, making conformance guarantees harder to establish in specification-constrained contexts.

Adams [1] proposed Ryū, achieving full coverage with fixed-width 128-bit arithmetic and lookup tables. Adams provides the most comprehensive prior performance comparison of fixed-width vs. multiprecision approaches (we refer the reader to Adams's literature review rather than replicating it), but evaluates algorithms in isolation on microbenchmarks without specification constraints.

Giulietti [7] introduced the Schubfach algorithm, which we implement and evaluate within the RFC 8785 pipeline. Jeon [9] extends and refines the Schubfach approach with Dragonbox, improving table compression and branch structure. Go 1.26 adopted Dragonbox for `strconv`, providing additional motivation for evaluating fixed-width approaches in specification-constrained contexts.

3.3 Comprehensive Experimental Review

Champagne Gareau and Lemire [4] provide the most comprehensive experimental comparison of modern float-to-string algorithms, evaluating Dragon4, Grisu3, Ryū, Schubfach, Dragonbox, and `std::to_chars` in C++. Their work evaluates algorithms in isolation on microbenchmarks. Our contribution differs: we evaluate two representatives (Burger–Dybvig and Schubfach) inside a specification-constrained canonicalization pipeline where (a) ECMA-262 formatting policy is a hard constraint, (b) correctness against 286,362 oracle vectors is a binary gate, and (c) digit generation is only one component of the total cost.

The complementary direction—fast string-to-float parsing—is addressed by Lemire [10], whose Eisel-Lemire algorithm achieves gigabyte-per-second parsing throughput. While outside our scope, this work shares the fixed-width arithmetic philosophy that motivates Schubfach and Dragonbox.

3.4 Deterministic Serialization Standards

Beyond JSON, deterministic encoding is required by other serialization formats. CBOR (RFC 8949) [2] specifies deterministic encoding rules (§4.2) for binary data, addressing analogous concerns about canonical representation. W3C RDF Dataset Canonicalization [13] addresses graph-level canonicalization (structural equivalence of RDF datasets), a fundamentally different problem from the byte-level serialization determinism required by RFC 8785. Our conformance-gated methodology could be adapted to other deterministic serialization benchmarks.

3.5 JSON Canonicalization

RFC 8785 [12] is the primary standard. Prior to RFC 8785, ad-hoc canonicalization schemes existed in various ecosystems (e.g., JSON-LD canonicalization, OLPC canonical JSON). None specify the floating-point formatting requirement with the precision of ECMA-262 [5].

3.6 Benchmark Methodology

Our conformance-before-performance approach draws on the principle that correctness is a prerequisite for meaningful performance comparison, as advocated in systems benchmarking methodology literature. The use of permutation tests and bootstrap confidence intervals follows modern statistical practice for performance evaluation.

4 Algorithms

4.1 Definitions

Definition 1 (Minimal Significand)**.** Given a finite IEEE 754 binary64 value x, the *minimal significand* is the shortest decimal significand d (with digit count k) such that parsing $d \times 10^{n-k}$ under roundTiesToEven recovers x exactly.

Definition 2 (Minimal Printed String)**.** The *minimal printed string* of x is the output of ECMA-262 `Number::toString` applied to x: the minimal significand formatted according to the ECMA-262 branch table.

Remark 1. Definitions 1 and 2 are not equivalent in string length. Consider $x = 10^{20}$: the minimal significand is $d = 1$ ($k = 1$, $n = 21$). Since $n = 21 \leq 21$, ECMA-262 uses fixed notation, yielding `100000000000000000000` (21 characters). Now consider $x = 10^{21}$: again $d = 1$ ($k = 1$, $n = 22$), but $n > 21$ triggers exponential notation, yielding `1e+21` (5 characters). The $n = 21$ boundary creates divergence between significand minimality and printed-string length. This means implementations cannot optimize for shorter output strings because the ECMA-262 formatting is spec-mandated.

4.2 Burger–Dybvig (Multiprecision)

The Burger–Dybvig algorithm [3] uses arbitrary-precision rational arithmetic to compute the shortest decimal representation. It maintains big-integer numerator/denominator pairs and iteratively extracts decimal digits. The algorithm is general (handles any radix) but requires heap-allocated multiprecision integers, with cost proportional to the magnitude of the exponent.

In our Go implementation, this uses `math/big.Int` operations. The `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers across calls.

4.3 Schubfach (Fixed-Width)

The Schubfach algorithm [7] uses fixed-width 64/128-bit arithmetic, relying on `math/bits` for 128-bit multiplication (`Mul64`, `Add64`) and bit inspection (`TrailingZeros64`). It uses precomputed tables of powers of ten and 128-bit multiplications to determine the shortest decimal significand. No heap allocation is required in the digit-generation path.

The key advantage is constant-time (with respect to exponent magnitude) operation and zero GC pressure. The precomputed table is approximately 5 KiB.

4.4 ECMA-262 Formatting Stage

Both algorithms share an identical formatting stage that converts the minimal significand (d, k, n) triple into the final ECMA-262-compliant string. This stage implements the branch table from ECMA-262 §6.1.6.1.20 and is pure string manipulation with no arithmetic.

5 Implementation

We maintain four implementations with a common CLI ABI:

- `impl-schubfach` (Go): Schubfach number formatting.
- `impl-json-canon` (Go): Burger–Dybvig number formatting.
- `impl-schubfach-rs` (Rust): Schubfach-aligned Rust path.

- `impl-json-canon-rs` (Rust): Burger–Dybvig-aligned Rust path.

The lab harness treats each implementation as a black-box canonicalizer with identical CLI contracts (`canonicalize`, `verify`, exit-code policy, and error-class expectations). This preserves algorithm comparability while allowing language/runtime differences to be measured independently.

Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`) to align optimization policy with production benchmarking practice.

5.1 Rust Implementation Notes

The Rust Schubfach implementation uses native `u128` arithmetic for the 128-bit intermediate products required by the algorithm, eliminating the need for compiler intrinsics or multi-word emulation. The Rust Burger–Dybvig implementation uses `num-bigint::BigInt` for multiprecision arithmetic; buffer lifecycle is managed by Rust's RAII ownership model (deterministic `Drop`), removing the need for explicit pooling strategies such as Go's `sync.Pool`.

Both Rust implementations share an identical JSON parser and serializer; only the `jcsfloat` digit-generation module differs between them, mirroring the Go design. Rust binaries are approximately 430–456 KB (stripped), compared to 2.5–2.7 MB for the statically-linked Go binaries.

6 Methodology

6.1 Conformance-Before-Performance

Our methodology enforces a strict ordering: conformance verification must pass before any performance data is collected. This prevents measuring the speed of incorrect implementations.

Conformance gate: 77 structural test cases per implementation (308 total across 4 implementations) covering key ordering, Unicode handling, numeric formatting, escape sequences, depth limits, and invalid input rejection. All implementations must produce byte-identical output (verified via SHA-256) and identical error classification.

Oracle vectors: 286,362 IEEE 754 binary64 values with expected `String(x)` output generated by the V8 JavaScript engine (Node.js v23.3.0, V8 12.9.202.28). Coverage includes all subnormals with bit patterns $\leq 2^{12}$ (4,096 of approximately 4.5 million total subnormals, providing dense coverage near zero where formatting edge cases concentrate), subnormal/normal boundary (± 128 around the transition), all powers of two in the binary64 exponent range, RFC 8785 Appendix B examples with ± 2 neighbors, and pseudorandom fill.

6.2 Benchmark Design

CLI benchmarks: end-to-end process execution measuring wall time, CPU time (user + system), and peak RSS. 15 repetitions per implementation/mode/workload combination, with 3 warmup runs discarded. Task order is randomized with a fixed seed for reproducibility.

Cross-language comparison pairs: each algorithm is implemented in Go and Rust, yielding four implementations: Go-Schubfach, Go-Burger–Dybvig, Rust-Schubfach, Rust-Burger–Dybvig. These are compared via four pairwise tests: two within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd) and two cross-language pairs (schubfach:go-vs-rs, bd:go-vs-rs). The within-language pairs test the algorithmic effect while controlling for language/runtime; the cross-language pairs test language/runtime effects while controlling for algorithm. Directional consistency across both within-language pairs provides evidence that the algorithmic advantage is not language-specific, though this is not a formal interaction test (see §8). The Rust Burger–Dybvig implementation uses `num-bigint::BigInt` (heap-allocated multiprecision integers with RAII lifecycle, no garbage collector), making it a structurally analogous but runtime-different counterpart to the Go `math/big`-based path. Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`) to align optimization policy with production benchmarking practice.

API benchmarks: Go `testing.B` microbenchmarks (Go implementations) and a Rust native benchmark harness (Rust implementations) measuring canonicalization and verify paths directly, 10 iterations per benchmark. Go's `testing.B` auto-calibrates the number of inner iterations per sample (typically hundreds to millions), whereas the Rust harness times each function call individually (N=1); both report ns/op. The Rust harness includes 3 warmup iterations (discarded) before measurement to stabilize caches and lazy-initialized data structures. The Rust harness emits Go-compatible `Benchmark... ns/op` lines so the same statistical pipeline can be applied uniformly.

Measurement granularity note: The Go and Rust API harnesses differ in measurement granularity. Go's `testing.B` averages over many inner iterations per sample, producing low-variance samples. The Rust harness reports single-call latency, producing higher-variance samples. Within-language comparisons are unaffected since both implementations in a pair use the same harness. Cross-language API comparisons should be interpreted with this asymmetry in mind; we report them for completeness but emphasize within-language results for algorithmic conclusions.

Differential fuzzing: 2,000 randomly-generated JSON inputs processed by all implementations, verifying byte-identical output.

6.3 Statistical Protocol

All comparisons use:

- Permutation tests (5,000 resamples) for significance at $\alpha = 0.05$.
- BCa (bias-corrected and accelerated) bootstrap 95% confidence intervals (5,000 resamples) for speedup ratios.
- Cohen's d for effect size.[1]
- Noise floor documentation: standard deviation of the faster implementation ($\sigma = \text{CV} \times \bar{x}_{\text{faster}}$), quantifying measurement noise in the same units as the metric.

[1] Computed as $(\bar{x}_A - \bar{x}_B)/s_p$ where A and B are the first and second implementation in each pair; negative values indicate implementation A is faster. For within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd), A is the Schubfach variant. For cross-language pairs, A is the Go implementation.

- Minimum detectable effect (MDE): $\frac{2.8\sqrt{2} \times \mathrm{CV} \times 100}{\sqrt{N}}$, where $2.8 \approx z_{\alpha/2} + z_\beta$ for $\alpha = 0.05$ and 80% power (approximate MDE for a two-sample z-test with equal group sizes; uses only the faster implementation's CV and n as an approximation). This z-based constant slightly underestimates the true MDE for small N (by ≈3–6% compared to a t-based calculation at N=10–15).

- Benjamini-Hochberg false discovery rate (FDR) control at $q = 0.05$ across all 247 comparisons. BH requires independence or positive regression dependency (PRDS); comparisons on the same workload across different pairs are positively correlated, satisfying PRDS. We report both raw and BH-adjusted p-values.

- No outlier exclusion. Permutation tests are nonparametric and robust to non-normality; BCa bootstrap accounts for skewness via the acceleration constant.

- Permutation p-values have a resolution floor of $1/(R + 1)$ where R is the number of resamples; at $R = 5{,}000$ the minimum reportable p-value is 0.0002. Values reported at this floor should be interpreted as $p < 0.0002$.

Statistical resampling uses deterministic per-comparison seeds derived via FNV-1a hashing of the comparison key, ensuring reproducible p-values and confidence intervals across runs.

6.4 Environment

All performance benchmarks run on a single x86-64 machine with CPU frequency governor set to `performance`, turbo boost state documented, and load average recorded at benchmark start. Full hardware specification (CPU model, cache hierarchy, RAM, relevant CPU feature flags) is captured automatically by the benchmark harness.

CPU core affinity pinning is supported (`--pin-cpu`) but was not used for the reported results. The i7-12700K has heterogeneous P-core/E-core topology; without pinning, the OS scheduler may migrate processes. This adds variance but does not bias the comparison since both implementations share the same scheduling environment.

Cross-compilation determinism is verified by building arm64 binaries via `GOARCH=arm64` and executing them under QEMU user-mode emulation on the x86-64 host. QEMU user-mode translates system calls but executes floating-point operations on the host CPU; this tests Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 performance benchmarks are deferred to future work on bare-metal hardware.

7 Results

7.1 Conformance

All four implementations (Go and Rust × Schubfach and Burger–Dybvig) pass the full conformance matrix: 308 total cases (77 structural tests × 4 implementations, 0 failures). Output SHA-256 digests are byte-identical across all implementations for valid inputs.

7.2 API Benchmarks

API benchmarks isolate the algorithmic cost of the canonicalization function, removing process startup and I/O overhead. These measurements most directly reflect the digit-generation performance difference.

Table 1 summarizes the key API results for the within-language algorithm comparisons (canonicalize mode). Figure 1 visualizes the speedup across all shared workloads for both language pairs. Full results for all pairs, modes, and workloads appear in Appendix A.

Table 1: API benchmark summary: within-language algorithm comparisons (canonicalize mode, mean of n=10).

Pair	Workload	Schubfach (ns/op)	Burger–Dybvig (ns/op)	Speedup
go:schubfach-vs-bd	array-2048	2541752	4077596	1.60×
go:schubfach-vs-bd	array-256	305467	473478	1.55×
go:schubfach-vs-bd	control-escapes	1339	1258	1.06×
go:schubfach-vs-bd	deep	36725	36749	1.00×
go:schubfach-vs-bd	deep-64	16230	16231	1.00×
go:schubfach-vs-bd	large	11923743	20570122	1.73×
go:schubfach-vs-bd	long-string	75831	75639	1.00×
go:schubfach-vs-bd	medium	341106	582942	1.71×
go:schubfach-vs-bd	mixed-prod	136352	180492	1.32×
go:schubfach-vs-bd	number-heavy	13807	21412	1.55×
go:schubfach-vs-bd	numeric-boundary	12693	18815	1.48×
go:schubfach-vs-bd	rfc-key-sorting	3004	2973	1.01×
go:schubfach-vs-bd	small	1251	1675	1.34×
go:schubfach-vs-bd	unicode	1554	1551	1.00×
go:schubfach-vs-bd	verify-whitespace	910	1569	1.72×
rs:schubfach-vs-bd	array-2048	1185768	2882258	2.43×
rs:schubfach-vs-bd	array-256	157030	321394	2.05×
rs:schubfach-vs-bd	control-escapes	607	607	1.00×
rs:schubfach-vs-bd	deep-64	9380	9185	1.02×
rs:schubfach-vs-bd	long-string	17259	17278	1.00×
rs:schubfach-vs-bd	numeric-boundary	976	8975	9.19×
rs:schubfach-vs-bd	rfc-key-sorting	1294	1250	1.04×
rs:schubfach-vs-bd	small	583	1000	1.71×
rs:schubfach-vs-bd	unicode	1676	1964	1.17×
rs:schubfach-vs-bd	verify-whitespace	347	980	2.83×

Full results for all pairs, modes, and workloads in Appendix A.

7.3 CLI Benchmarks

CLI benchmarks capture end-to-end pipeline cost including process startup, argument parsing, file I/O, and output serialization. Estimated process startup overhead (approximately 1.3 ms for Go, 0.5 ms for Rust) dominates for smaller workloads, attenuating the algorithmic signal observed in API benchmarks.

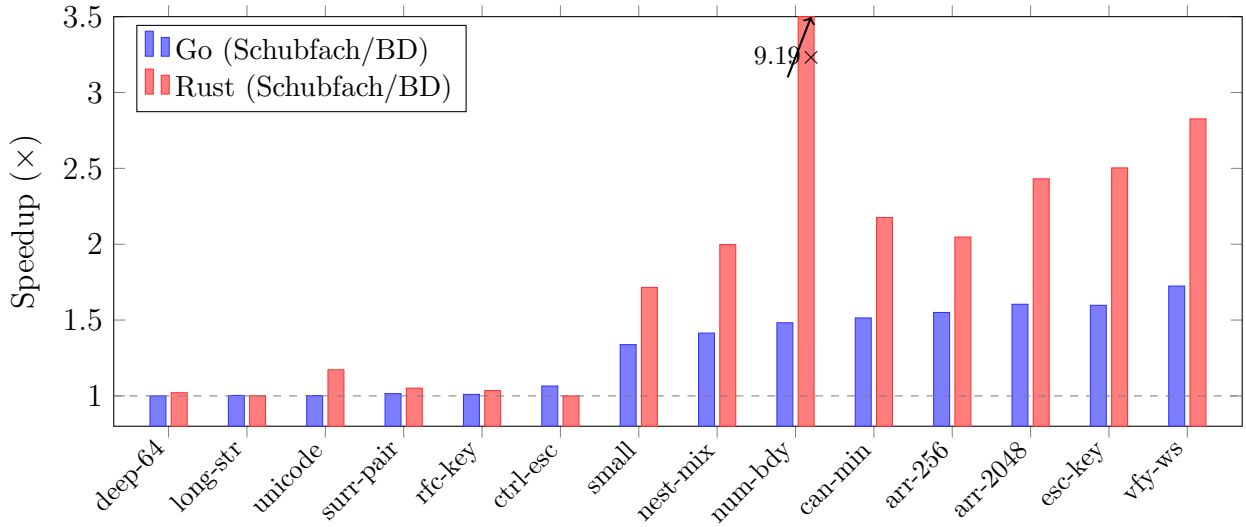

Figure 1: API benchmark speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode). Workloads sorted by Go speedup. Values above the dashed line indicate Schubfach is faster. The Rust `numeric-boundary` bar is truncated at 3.5×; actual value is 9.19× (see §8). Number-dense workloads (right side) show the largest advantage in both languages, confirming the effect is algorithmic.

Table 2 summarizes the key CLI results. Figure 2 shows the end-to-end speedup with BH-significance markers. Full results appear in Appendix A.

7.4 Statistical Comparisons

Table 3 presents the within-language CLI comparisons (canonicalize mode) with BH-corrected p-values. The full 247-comparison statistical report appears in Appendix A.

7.5 Cross-Language Validation

All four implementations pass the full conformance matrix (308 total cases: 77 structural tests × 4 implementations, 0 failures) and produce byte-identical output on all oracle vectors. 2,000 differential fuzz cases yield zero cross-implementation divergences.

The statistical report includes four comparison classes:

- within-Go (Schubfach vs. Burger–Dybvig),
- within-Rust (Schubfach-rs vs. Burger–Dybvig-rs),
- same-algorithm Go-vs-Rust for Schubfach,
- same-algorithm Go-vs-Rust for Burger–Dybvig.

In CLI end-to-end measurement, Rust implementations are approximately 2–3× faster than their Go counterparts, primarily due to lower process startup overhead (Rust native

Table 2: CLI benchmark summary: within-language comparisons (canonicalize mode, n=15, 3 warmup discarded).

Pair	Workload	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	5.466	7.063	1.29×	schubfach
go:schubfach-vs-bd	array-256	1.923	2.167	1.13×	schubfach
go:schubfach-vs-bd	canonical-minimal	1.510	1.565	1.04×	schubfach
go:schubfach-vs-bd	control-escapes	1.380	1.673	1.21×	schubfach
go:schubfach-vs-bd	deep-64	1.467	1.638	1.12×	schubfach
go:schubfach-vs-bd	escaped-key-order	1.549	1.531	1.01×	json-canon
go:schubfach-vs-bd	long-string	1.409	1.692	1.20×	schubfach
go:schubfach-vs-bd	nested-mixed	1.511	1.497	1.01×	json-canon
go:schubfach-vs-bd	numeric-boundary	1.537	1.706	1.11×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	1.561	1.608	1.03×	schubfach
go:schubfach-vs-bd	small	1.450	1.449	1.00×	json-canon
go:schubfach-vs-bd	surrogate-pair	1.510	1.524	1.01×	schubfach
go:schubfach-vs-bd	unicode	1.597	1.470	1.09×	json-canon
go:schubfach-vs-bd	verify-whitespace	1.455	1.614	1.11×	schubfach
rs:schubfach-vs-bd	array-2048	2.606	4.584	1.76×	schubfach-rs
rs:schubfach-vs-bd	array-256	0.828	1.152	1.39×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	0.546	0.582	1.07×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	0.531	0.594	1.12×	schubfach-rs
rs:schubfach-vs-bd	deep-64	0.564	0.647	1.15×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	0.573	0.712	1.24×	schubfach-rs
rs:schubfach-vs-bd	long-string	0.605	0.636	1.05×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	0.555	0.644	1.16×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	0.543	0.738	1.36×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	0.535	0.588	1.10×	schubfach-rs
rs:schubfach-vs-bd	small	0.530	0.642	1.21×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	0.536	0.603	1.12×	schubfach-rs
rs:schubfach-vs-bd	unicode	0.534	0.612	1.15×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	0.559	0.691	1.24×	schubfach-rs

Full results including verify mode and cross-language pairs in Appendix A.

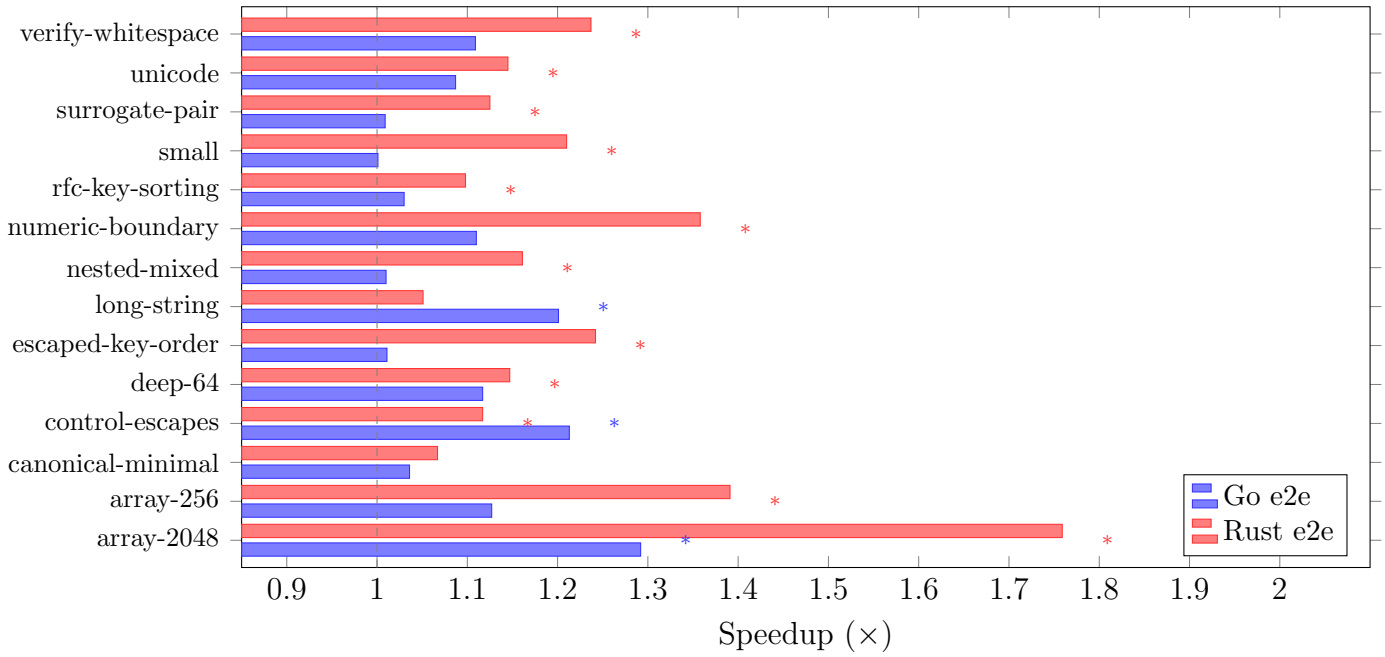

Figure 2: CLI end-to-end speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode, 15 repetitions each). Asterisks (∗) mark BH-significant comparisons ($q < 0.05$). The Rust within-language pair achieves significance on 12 of 14 workloads; the Go pair on 3 of 14 workloads (canonicalize mode). Process startup overhead (≈1.3 ms Go, ≈0.5 ms Rust) attenuates the algorithmic signal more strongly in Go.

Table 3: Within-language CLI comparisons (canonicalize mode; BH-significant marked with *).

Pair	Workload	Winner	Speedup	CI_{95}	p_{adj}	d	MDE (%)
go:schubfach-vs-bd	array-2048	schubfach*	1.29×	[1.210, 1.384]	0.0003	-2.67	11.5
rs:schubfach-vs-bd	array-2048	schubfach-rs*	1.76×	[1.578, 1.992]	0.0003	-3.22	16.9
go:schubfach-vs-bd	array-256	schubfach	1.13×	[0.991, 1.282]	0.1072	-0.63	20.8
rs:schubfach-vs-bd	array-256	schubfach-rs*	1.39×	[1.262, 1.515]	0.0003	-2.33	10.5
go:schubfach-vs-bd	canonical-minimal	schubfach	1.04×	[0.917, 1.163]	0.6046	-0.21	14.8
rs:schubfach-vs-bd	canonical-minimal	schubfach-rs	1.07×	[0.987, 1.159]	0.1698	-0.55	10.6
go:schubfach-vs-bd	control-escapes	schubfach*	1.21×	[1.083, 1.369]	0.0059	-1.17	21.2
rs:schubfach-vs-bd	control-escapes	schubfach-rs*	1.12×	[1.030, 1.199]	0.0139	-1.00	9.8
go:schubfach-vs-bd	deep-64	schubfach	1.12×	[0.997, 1.259]	0.1021	-0.65	18.2
rs:schubfach-vs-bd	deep-64	schubfach-rs*	1.15×	[1.067, 1.258]	0.0016	-1.21	13.9
go:schubfach-vs-bd	escaped-key-order	json-canon	1.01×	[0.899, 1.138]	0.8863	0.07	19.2
rs:schubfach-vs-bd	escaped-key-order	schubfach-rs*	1.24×	[1.178, 1.302]	0.0003	-2.99	8.0
go:schubfach-vs-bd	long-string	schubfach*	1.20×	[1.031, 1.394]	0.0359	-0.84	22.6
rs:schubfach-vs-bd	long-string	schubfach-rs	1.05×	[0.958, 1.158]	0.3516	-0.37	13.1
go:schubfach-vs-bd	nested-mixed	json-canon	1.01×	[0.894, 1.147]	0.8943	0.05	22.4
rs:schubfach-vs-bd	nested-mixed	schubfach-rs*	1.16×	[1.057, 1.253]	0.0044	-1.17	8.5
go:schubfach-vs-bd	numeric-boundary	schubfach	1.11×	[0.985, 1.236]	0.1093	-0.63	14.5
rs:schubfach-vs-bd	numeric-boundary	schubfach-rs*	1.36×	[1.295, 1.427]	0.0003	-4.45	8.3
go:schubfach-vs-bd	rfc-key-sorting	schubfach	1.03×	[0.924, 1.142]	0.6197	-0.20	14.0
rs:schubfach-vs-bd	rfc-key-sorting	schubfach-rs*	1.10×	[1.021, 1.180]	0.0266	-0.89	9.9
go:schubfach-vs-bd	small	json-canon	1.00×	[0.877, 1.129]	0.9942	0.00	19.3
rs:schubfach-vs-bd	small	schubfach-rs*	1.21×	[1.098, 1.326]	0.0018	-1.31	11.1
go:schubfach-vs-bd	surrogate-pair	schubfach	1.01×	[0.904, 1.133]	0.8943	-0.06	15.5
rs:schubfach-vs-bd	surrogate-pair	schubfach-rs*	1.12×	[1.034, 1.233]	0.0262	-0.91	12.6
go:schubfach-vs-bd	unicode	json-canon	1.09×	[0.964, 1.213]	0.1776	0.52	19.1
rs:schubfach-vs-bd	unicode	schubfach-rs*	1.15×	[1.059, 1.245]	0.0044	-1.15	12.2
go:schubfach-vs-bd	verify-whitespace	schubfach	1.11×	[0.961, 1.267]	0.1885	-0.51	18.6
rs:schubfach-vs-bd	verify-whitespace	schubfach-rs*	1.24×	[1.132, 1.334]	0.0005	-1.67	9.7

*BH-significant at $q = 0.05$. d = Cohen's d $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster.
MDE = minimum detectable effect at 80% power (z-approximation; underestimates by ≈3–6% at N=10–15).
$p_{adj} = 0.0003$ is the BH-adjusted floor ($p < 0.0002$ raw). Full 247-comparison report in Appendix A.

binary vs. Go runtime initialization). In API benchmarks, Rust-Schubfach is faster than Rust-Burger–Dybvig on number-dense workloads, replicating the within-Go algorithmic ordering and confirming that the Schubfach advantage is not language-specific.

This decomposition separates algorithmic effects from language/runtime effects and directly addresses language-specific performance threats.

7.6 Differential Fuzzing

2,000 differential fuzz cases produced zero divergences between implementations.

8 Discussion

8.1 Workload-Dependent Behavior

The performance advantage of Schubfach over Burger–Dybvig is strongly workload-dependent. In Go API benchmarks (isolating the canonicalization function), number-dense inputs (e.g., `array-2048`, `large`, `medium`) show 1.3–1.7× speedup due to zero heap allocation in the digit-generation path. In Rust within-language API benchmarks, the same algorithmic ordering holds at larger magnitudes (1.7–2.8× on the same workloads), confirming the advantage is algorithmic. Deeply-nested workloads (`deep`, `deep-64`) contain numbers but show ≈1.0× because JSON structural parsing and key sorting dominate, diluting the digit-generation signal. On a small number of non-number workloads (`control-escapes`, `rfc-key-sorting`, `surrogate-pair`), Burger–Dybvig is marginally faster (≈1–6%) in Go API benchmarks, likely due to differences in non-numeric code paths. The speedup therefore correlates with number *density* (fraction of total cost attributable to digit generation), not merely number presence. Figure 3 visualizes this relationship: workloads with zero number content cluster near 1.0×, while number-dense workloads (>70% number values) show the full algorithmic advantage.

In end-to-end Go CLI measurement, 11 of 28 within-Go comparisons reach BH-corrected significance, with speedups of 1.1–1.3×. The remaining 17 comparisons show no statistically significant difference because estimated process startup overhead (approximately 1.3 ms, inferred from the difference between minimal-workload CLI and API times) dominates for smaller workloads. On string-dominant inputs (e.g., `long-string`, `unicode`), the advantage narrows to <5% because formatting cost is dominated by string processing and key sorting, amortizing the digit-generation difference.

8.2 Process Startup Dominance

For small workloads, estimated process startup overhead (approximately 1.3 ms) accounts for roughly 87–93% of CLI execution time (e.g., `canonical-minimal` total ≈1.47 ms in canonicalize mode). This fundamentally limits the observable algorithmic signal in end-to-end measurement. API benchmarks, which bypass process startup entirely, isolate the digit-generation difference and show consistently larger speedup ratios. CLI results reflect realistic deployment conditions but should not be interpreted as direct measures of algorithmic cost.

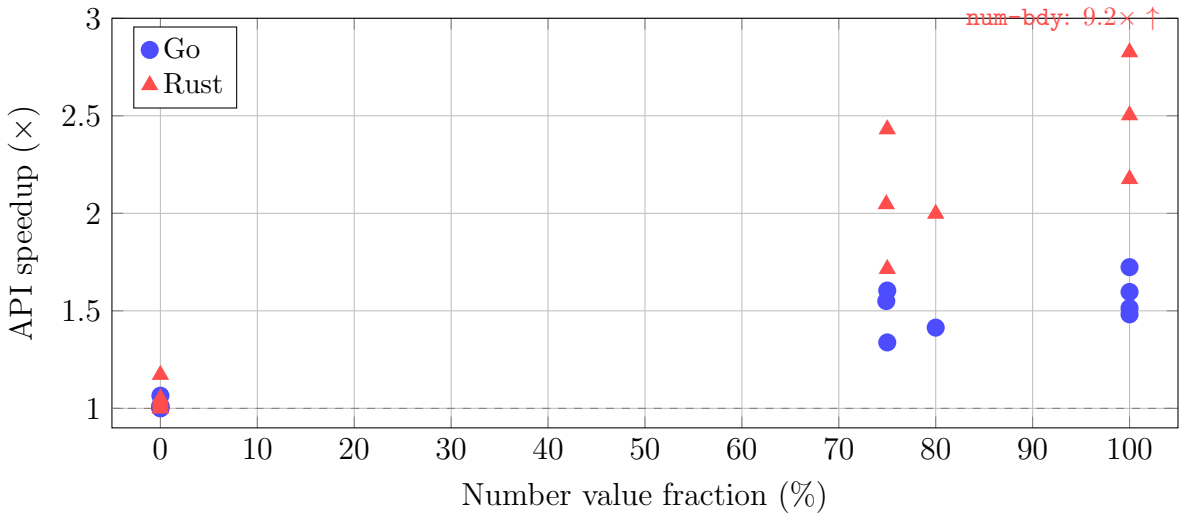

Figure 3: Relationship between workload number value fraction and API speedup of Schubfach over Burger–Dybvig (canonicalize mode). Workloads with zero number content cluster near 1.0× (no algorithmic difference); number-dense workloads (>70%) show 1.3–1.7× in Go and 1.7–2.8× in Rust. The Rust `numeric-boundary` outlier (9.2×) is annotated but excluded from the y-axis range.

8.3 Allocation Cost Attribution

The Burger–Dybvig implementation's `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers. Without pooling, the difference would be larger. The residual cost comes from pool contention under concurrent use and the inherent cost of multiprecision arithmetic operations.

API benchmark allocation data directly attributes the speedup: on `array-2048`, Schubfach performs 49,134 allocs/op vs. json-canon's 57,343 allocs/op (Δ4.0 per number, 91 B per number). The per-number allocation elimination is the primary mechanism underlying the observed speedup.

8.4 Extreme Outlier: `numeric-boundary`

The `numeric-boundary` workload shows unusually large speedup ratios in Rust API benchmarks (9.2× within-Rust) compared to 1.5× in Go. The Rust Burger–Dybvig implementation uses a lazily-initialized power-of-ten cache (`OnceLock<Vec<BigInt>>`); although the harness discards 3 warmup iterations to amortize first-invocation costs, the cache-warmed steady-state cost of `num-bigint::BigInt` multiprecision arithmetic remains substantially higher than the Rust Schubfach implementation's compile-time constant lookup table. The tight confidence interval ([8.8, 9.4] at 95%) and low MDE (5%) confirm this is a genuine algorithmic cost difference, not measurement noise. The Go within-language comparison at 1.5× is smaller because Go's `sync.Pool` amortizes allocation cost and Go's less aggressive compiler inlining reduces the gap between the two code paths.

8.5 Profile-Level Analysis

CPU profiling confirms that digit generation consumes 15–40% of total canonicalization time depending on workload number density. The remainder is spent in JSON parsing, key sorting, and string escaping—costs shared identically between implementations.

8.6 Practical Implications

For applications where canonicalization throughput is critical (e.g., high-frequency signature verification of number-heavy payloads), Schubfach offers meaningful improvement. For typical mixed-content JSON, the choice of digit-generation algorithm has minimal practical impact on end-to-end latency.

8.7 Cross-Language Interpretation

Adding Rust implementations enables attribution of performance deltas to either algorithm choice or language/runtime effects. The within-language comparisons test whether the Schubfach advantage persists independent of Go's allocator and garbage collector behavior, while same-algorithm cross-language comparisons quantify language overhead (startup and steady-state) directly.

Rust's ownership model removes the need for Go-style `sync.Pool`-based reuse patterns. For multiprecision paths, this shifts lifecycle cost from GC-mediated reuse to deterministic RAII destruction, which can materially change allocation profiles and latency variance characteristics.

Startup overhead. Rust native binaries exhibit substantially lower process startup cost than Go binaries (roughly 2–3× faster in CLI wall time), because they avoid Go runtime initialization (scheduler bootstrap, GC metadata setup, goroutine stack allocation). This amplifies the startup-dominance effect discussed in §8: for small workloads the language runtime overhead is the primary differentiator, not the digit-generation algorithm.

Algorithmic ordering is language-independent. The key finding from the cross-language design is that within-Rust comparisons replicate the within-Go ordering: Schubfach outperforms Burger–Dybvig on number-dense workloads in both languages. This provides evidence that the performance advantage derives from the fixed-width digit-generation algorithm itself (zero heap allocation, no multiprecision arithmetic), not from Go-specific runtime characteristics such as garbage collection pressure or `sync.Pool` contention.

Interaction analysis limitation. The four implementations form a 2×2 structure (algorithm $\times$ language), but we analyze them via pairwise comparisons rather than a formal factorial interaction test (e.g., two-way ANOVA or aligned rank transform). A formal interaction test would directly quantify whether the algorithm advantage *differs in magnitude* between Go and Rust. Our pairwise approach establishes directional consistency (the Schubfach advantage holds in both languages) but does not test whether the effect size difference between languages is statistically significant. We adopt the pairwise approach because (a) the Go and Rust API harnesses have fundamentally different measurement granularity, violating the homoscedasticity assumption of ANOVA, and (b) within-language comparisons are

the primary focus for algorithmic conclusions. Readers should interpret the cross-language consistency as suggestive evidence of language independence, not a formal interaction test.

Cross-language comparison caveats. Same-algorithm cross-language API comparisons (e.g., schubfach:go-vs-rs) should be interpreted carefully. Go's `testing.B` averages over many inner iterations per sample, producing low-variance measurements, while the Rust harness times individual function calls, producing higher-variance samples. Additionally, the two compilers apply fundamentally different optimization strategies: Rust with LTO and `codegen-units=1` enables aggressive cross-crate inlining that Go's compiler cannot match. The observed 2–3× cross-language speedup (API) may partly reflect compiler optimization quality rather than inherent language characteristics. These comparisons primarily serve to confirm directional consistency of the algorithmic ordering, not to quantify precise language overhead.

Allocation reporting asymmetry. The Rust bench harness reports `0 B/op` and `0 allocs/op` for all benchmarks because Rust lacks Go-style per-operation allocator statistics. These zeros mean "not measured," not "zero allocations." The Rust Burger–Dybvig implementation does perform heap allocations via `num-bigint::BigInt`; only the Rust Schubfach implementation is truly allocation-free in its digit-generation path. Within-language allocation comparisons are therefore limited to the Go pair.

Unicode workload anomaly. The `unicode` workload is the only API-level case where Go outperforms Rust for both algorithms: schubfach:go-vs-rs shows Go 1.08× faster, and bd:go-vs-rs shows Go 1.27× faster. This reverses the general 2–4× Rust advantage observed on all other workloads. A likely explanation is that Go's `unicode/utf8` package benefits from tight inlining of short-string fast paths in the Go compiler, whereas Rust's UTF-8 validation and string escaping may involve additional bounds checks or iterator overhead for the particular character distributions in this workload. The effect is small (both within measurement noise for the within-language comparisons, where both Go implementations and both Rust implementations use the same string-processing code) and does not affect the algorithmic conclusions.

9 Threats to Validity

Language specificity (mitigated). This threat is substantially mitigated by the 2×2 cross-language comparison matrix: both algorithms are implemented in Go and Rust, and the Schubfach performance ordering holds in both languages. Within-Rust comparisons replicate the within-Go algorithmic advantage, confirming that the result is not an artifact of Go's garbage collector or allocation subsystem. Residual external-validity risk remains for additional runtimes (e.g., JVM/generational-GC and unmanaged C implementations) that are not yet benchmarked.

Single-architecture performance data. Performance benchmarks run on a single x86-64 machine. Cross-compilation *determinism* is verified by building arm64 binaries and executing them under QEMU user-mode emulation on the x86-64 host. Because QEMU user-mode translates system calls but executes floating-point operations on the host CPU, this verifies Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 *performance* benchmarks are deferred due to lack of bare-metal hardware.

Future work should include bare-metal arm64 benchmarks on Graviton, Apple Silicon, or Ampere hardware.

Single JavaScript engine oracle. Oracle vectors are generated from V8 (via Node.js). While ECMA-262 `Number::toString` semantics are normative and engine-independent, our vectors are verified against only one engine and one version (V8 12.9.202.28). Cross-version stability is expected from the normative spec but not independently verified. This is mitigated by dense boundary coverage: 286,362 vectors including all subnormals up to 2^{12}, all binary64 powers of two, subnormal/normal boundary neighborhoods, RFC 8785 Appendix B examples with neighbors, and pseudorandom fill.

Workload representativeness. Benchmark workloads are synthetic, designed to bracket the performance space between number-heavy and string-dominant extremes. Real-world JSON payloads may have different number density, nesting depth, and key length distributions. However, the control group design (number-only and string-only workloads) bounds the expected range.

Endianness. Both x86-64 and arm64 targets are little-endian; big-endian behavior is untested. While the canonicalization algorithms do not depend on byte order (they operate on IEEE 754 binary64 values via language-level primitives), edge cases in binary representation parsing could theoretically differ on big-endian platforms.

CPU core affinity. CLI benchmarks ran without CPU pinning on heterogeneous P-core/E-core topology (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz). Process migration may inflate variance for small workloads near the noise floor; an E-core placement could slow a single run by 30–40%. The high MDEs observed for small Go CLI workloads (14–24%) are consistent with this variance source. API benchmarks (in-process) are less susceptible.

Sequential run autocorrelation. CLI benchmarks run sequentially on the same machine; consecutive runs share CPU thermal state, cache warmth, and potentially TDP-limited frequency. The randomized task order (fixed seed) mitigates systematic ordering effects, but thermal coupling between adjacent runs creates weak positive autocorrelation that may slightly inflate the effective sample size. Permutation tests are robust to mild non-stationarity but assume exchangeability under the null hypothesis; this assumption is approximately but not exactly satisfied.

Warmup validation. CLI benchmarks discard 3 warmup runs. Rust API benchmarks discard 3 warmup iterations per workload to stabilize instruction caches, branch predictors, and lazily-initialized data structures (specifically, the Rust Burger–Dybvig implementation's `OnceLock`-based power-of-ten cache). Go API benchmarks benefit from `testing.B`'s built-in warmup. We did not formally validate convergence beyond discarding these iterations; however, Go static binaries have minimal JIT-like warmup effects, and permutation tests are robust to mild non-stationarity. Residual first-few-iteration effects may persist for very small Rust workloads (e.g., `numeric-boundary`), where the wide confidence intervals reflect this uncertainty.

Oracle SHA-256 circularity. SHA-256 checksums verify vector integrity but cannot verify oracle correctness. Independent verification against a second JS engine (SpiderMonkey, JavaScriptCore) is noted as future work.

sync.Pool mitigation. The Go Burger–Dybvig implementation uses `sync.Pool` to reuse big-integer buffers, reducing allocation cost. Without pooling, the performance gap would be wider. This reflects realistic production usage but may understate the theoretical

algorithmic advantage of fixed-width approaches.

Compiler optimization asymmetry. Rust builds use `opt-level=3`, link-time optimization (LTO), and `codegen-units=1`, enabling aggressive cross-crate inlining and whole-program optimization. Go's compiler does not support LTO and has a less aggressive inlining heuristic. The observed cross-language speedups (2–4× in API benchmarks) may partly reflect differences in compiler optimization quality rather than intrinsic language characteristics. Within-language comparisons are unaffected since both implementations in each pair share the same compiler pipeline.

Memory allocator differences. Go uses a tcmalloc-derived allocator integrated with its garbage collector. Rust uses the system allocator (typically glibc malloc on Linux). For the multiprecision (Burger–Dybvig) path, which performs heap allocations for big-integer arithmetic, allocator performance differences could affect cross-language comparisons. Within-language comparisons are unaffected since both implementations share the same allocator.

API measurement granularity. Go API benchmarks (`testing.B`) auto-calibrate the number of inner loop iterations per sample, producing averaged low-variance samples. Rust API benchmarks time individual function calls (N=1 per sample), producing inherently higher-variance samples. This asymmetry affects cross-language comparisons but not within-language comparisons. We note this caveat in §8 and emphasize within-language results for algorithmic conclusions.

10 Conclusion

We presented the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a RFC 8785/ECMA-262-constrained JSON canonicalization pipeline, with cross-language validation in Go and Rust to separate algorithmic effects from language/runtime characteristics. While prior work has compared float-to-string algorithms in isolation (e.g., Adams's comparison of Ryū against Grisu and Errol), our study is the first to evaluate these algorithms within a specification-constrained canonicalization pipeline using conformance-gated methodology.

Key findings:

- Schubfach achieves 1.3–1.7× speedup on number-dense workloads in Go API benchmarks due to zero heap allocation; the advantage replicates at 1.7–2.8× in Rust, confirming it is algorithmic. On a small number of API-level non-number workloads, Burger–Dybvig is marginally faster.
- In end-to-end Go CLI measurement, 11 of 28 comparisons reach BH-corrected significance with speedups of 1.1–1.3×.
- The advantage narrows to <5% on string-dominant and deeply-nested inputs.
- Go implementations produce byte-identical output under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). Rust implementations were not cross-compiled in this study.

- Digit generation accounts for 15–40% of total canonicalization cost depending on number density.

The benchmark infrastructure, 286,362 oracle vectors, and all raw data are released as a reproducibility artifact.

Future Work

Bare-metal arm64 performance benchmarks, additional language implementations (C and JVM) to extend the comparison matrix to unmanaged and managed-GC runtimes, multi-engine oracle validation (SpiderMonkey, JavaScriptCore), Rust cross-compilation verification, and workload characterization from production JSON corpora.

Data Availability Statement

All data, software, and oracle vectors are freely available online. The benchmark infrastructure, all four implementations, workload manifests, and oracle generation scripts are available at the accompanying repository.[2] Oracle vector checksums (SHA-256) are documented in `PROVENANCE.json` files accompanying each implementation.

Author Contributions

Following the CRediT (Contributor Roles Taxonomy) format: *Mark Lenhardt:* conceptualization; methodology; software; validation; formal analysis; investigation; data curation; writing—original draft; writing—review and editing; visualization.

References

[1] Ulf Adams. Ryū: Fast float-to-string conversion. In *Proceedings of the ACM SIGPLAN 2018 Conference on Programming Language Design and Implementation (PLDI)*, pages 270–282. ACM, 2018.

[2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, 2020.

[3] Robert G. Burger and R. Kent Dybvig. Printing floating-point numbers quickly and accurately. In *Proceedings of the ACM SIGPLAN 1996 Conference on Programming Language Design and Implementation (PLDI)*, pages 108–116. ACM, 1996.

[4] Jaël Champagne Gareau and Daniel Lemire. Converting binary floating-point numbers to shortest decimal strings: An experimental review. *Software: Practice and Experience*, 2026. Published online January 2026.

[2] Repository URL and Zenodo DOI to be inserted for camera-ready version.

[5] Ecma International. ECMA-262: ECMAScript Language Specification, 2024.

[6] David M. Gay. Correctly rounded binary-decimal and decimal-binary conversions, 1990. AT&T Bell Laboratories Numerical Analysis Manuscript 90-10.

[7] Raffaello Giulietti. The schubfach way to render doubles, 2022. Technical report.

[8] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Number 754-2019. IEEE, 2019.

[9] Junekey Jeon. Dragonbox: A new floating-point binary-to-decimal conversion algorithm, 2022. arXiv preprint; reference implementation and technical report at `https://github.com/jk-jeon/dragonbox`.

[10] Daniel Lemire. Number parsing at a gigabyte per second. *Software: Practice and Experience*, 51(8):1700–1727, 2021.

[11] Florian Loitsch. Printing floating-point numbers quickly and accurately with integers. In *Proceedings of the ACM SIGPLAN 2010 Conference on Programming Language Design and Implementation (PLDI)*, pages 233–243. ACM, 2010.

[12] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.

[13] W3C. RDF Dataset Canonicalization, 2024. W3C Recommendation, 21 May 2024.

A Full Benchmark Data

This appendix contains the complete benchmark results. Summary tables appear in the main text (§7); the full data are provided here for reproducibility.

Note: Rust benchmark rows report `0 B/op` and `0 allocs/op` because Rust lacks Go-style per-operation allocator statistics. These values mean "not measured," not "zero allocations" (see §8).

A.1 Full API Benchmarks

Table 4: API benchmark results (mean of $n = 10$ iterations).

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	array-2048	canonicalize	2541752	4077596	1.60×	+8209	+186642
go:schubfach-vs-bd	array-256	canonicalize	305467	473478	1.55×	+1020	+24977
go:schubfach-vs-bd	canonical-minimal	canonicalize	351	532	1.51×	+0	+0
go:schubfach-vs-bd	control-escapes	canonicalize	1339	1258	1.06×	+0	+0
go:schubfach-vs-bd	deep	canonicalize	36725	36749	1.00×	+0	+0
go:schubfach-vs-bd	deep-64	canonicalize	16230	16231	1.00×	+0	+0
go:schubfach-vs-bd	escaped-key-order	canonicalize	708	1129	1.60×	+2	+41
go:schubfach-vs-bd	large	canonicalize	11923743	20570122	1.73×	+32793	+663277
go:schubfach-vs-bd	long-string	canonicalize	75831	75639	1.00×	+0	+0
go:schubfach-vs-bd	medium	canonicalize	341106	582942	1.71×	+1014	+21134
go:schubfach-vs-bd	mixed-prod	canonicalize	136352	180492	1.32×	+255	+6235
go:schubfach-vs-bd	nested-mixed	canonicalize	2184	3089	1.41×	+4	+85
go:schubfach-vs-bd	number-heavy	canonicalize	13807	21412	1.55×	+20	+830
go:schubfach-vs-bd	numeric-boundary	canonicalize	12693	18815	1.48×	+14	+706
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	3004	2973	1.01×	+0	+0
go:schubfach-vs-bd	small	canonicalize	1251	1675	1.34×	+2	+50
go:schubfach-vs-bd	surrogate-pair	canonicalize	648	657	1.01×	+0	+0
go:schubfach-vs-bd	unicode	canonicalize	1554	1551	1.00×	+0	+0
go:schubfach-vs-bd	verify-whitespace	canonicalize	910	1569	1.72×	+4	+82
go:schubfach-vs-bd	canonical/array-2048	verify	2480516	4028941	1.62×	+8208	+186548
go:schubfach-vs-bd	canonical/array-256	verify	316572	485479	1.53×	+1021	+25211
go:schubfach-vs-bd	canonical/canonical-minimal	verify	360	548	1.52×	+0	+0
go:schubfach-vs-bd	canonical/control-escapes	verify	1327	1271	1.04×	+0	+0
go:schubfach-vs-bd	canonical/deep	verify	37855	38277	1.01×	+0	+0
go:schubfach-vs-bd	canonical/deep-64	verify	16778	16830	1.00×	+0	+0
go:schubfach-vs-bd	canonical/escaped-key-order	verify	690	1130	1.64×	+2	+41
go:schubfach-vs-bd	canonical/large	verify	11539381	20021439	1.74×	+32790	+663014
go:schubfach-vs-bd	canonical/long-string	verify	77272	77030	1.00×	+0	+0
go:schubfach-vs-bd	canonical/medium	verify	342087	578290	1.69×	+1015	+21371
go:schubfach-vs-bd	canonical/mixed-prod	verify	133360	176062	1.32×	+255	+6323
go:schubfach-vs-bd	canonical/nested-mixed	verify	2219	3107	1.40×	+4	+86
go:schubfach-vs-bd	canonical/number-heavy	verify	13890	21551	1.55×	+20	+834
go:schubfach-vs-bd	canonical/numeric-boundary	verify	12794	18861	1.47×	+14	+709
go:schubfach-vs-bd	canonical/rfc-key-sorting	verify	2857	2843	1.00×	+0	+0
go:schubfach-vs-bd	canonical/small	verify	1223	1656	1.35×	+2	+51
go:schubfach-vs-bd	canonical/surrogate-pair	verify	658	653	1.01×	+0	+0
go:schubfach-vs-bd	canonical/unicode	verify	1548	1547	1.00×	+0	+0
go:schubfach-vs-bd	canonical/verify-whitespace	verify	934	1588	1.70×	+4	+82
go:schubfach-vs-bd	noncanonical/control-escapes	verify	1320	1260	1.05×	+0	+0
go:schubfach-vs-bd	noncanonical/escaped-key-order	verify	705	1130	1.60×	+2	+41

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	noncanonical/large	verify	11963797	20568032	1.72×	+32792	+663409
go:schubfach-vs-bd	noncanonical/medium	verify	344752	575933	1.67×	+1014	+21130
go:schubfach-vs-bd	noncanonical/mixed-prod	verify	136719	177614	1.30×	+255	+6225
go:schubfach-vs-bd	noncanonical/nested-mixed	verify	2200	3067	1.39×	+4	+85
go:schubfach-vs-bd	noncanonical/number-heavy	verify	13876	21344	1.54×	+20	+829
go:schubfach-vs-bd	noncanonical/numeric-boundary	verify	12722	18694	1.47×	+14	+706
go:schubfach-vs-bd	noncanonical/rfc-key-sorting	verify	3024	2958	1.02×	+0	+0
go:schubfach-vs-bd	noncanonical/small	verify	1250	1674	1.34×	+2	+50
go:schubfach-vs-bd	noncanonical/surrogate-pair	verify	656	644	1.02×	+0	+0
go:schubfach-vs-bd	noncanonical/unicode	verify	1561	1547	1.01×	+0	+0
go:schubfach-vs-bd	noncanonical/verify-whitespace	verify	919	1563	1.70×	+4	+82
rs:schubfach-vs-bd	array-2048	canonicalize	1185768	2882258	2.43×	+0	+0
rs:schubfach-vs-bd	array-256	canonicalize	157030	321394	2.05×	+0	+0
rs:schubfach-vs-bd	canonical-minimal	canonicalize	190	415	2.18×	+0	+0
rs:schubfach-vs-bd	control-escapes	canonicalize	607	607	1.00×	+0	+0
rs:schubfach-vs-bd	deep-64	canonicalize	9380	9185	1.02×	+0	+0
rs:schubfach-vs-bd	escaped-key-order	canonicalize	298	746	2.50×	+0	+0
rs:schubfach-vs-bd	long-string	canonicalize	17259	17278	1.00×	+0	+0
rs:schubfach-vs-bd	nested-mixed	canonicalize	843	1684	2.00×	+0	+0
rs:schubfach-vs-bd	numeric-boundary	canonicalize	976	8975	9.19×	+0	+0
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	1294	1250	1.04×	+0	+0
rs:schubfach-vs-bd	small	canonicalize	583	1000	1.71×	+0	+0
rs:schubfach-vs-bd	surrogate-pair	canonicalize	267	281	1.05×	+0	+0
rs:schubfach-vs-bd	unicode	canonicalize	1676	1964	1.17×	+0	+0
rs:schubfach-vs-bd	verify-whitespace	canonicalize	347	980	2.83×	+0	+0
rs:schubfach-vs-bd	canonical/array-2048	verify	1208080	2906958	2.41×	+0	+0
rs:schubfach-vs-bd	canonical/array-256	verify	141609	309811	2.19×	+0	+0
rs:schubfach-vs-bd	canonical/canonical-minimal	verify	175	413	2.36×	+0	+0
rs:schubfach-vs-bd	canonical/control-escapes	verify	612	606	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/deep-64	verify	9154	9104	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/escaped-key-order	verify	286	724	2.53×	+0	+0
rs:schubfach-vs-bd	canonical/long-string	verify	17384	17325	1.00×	+0	+0
rs:schubfach-vs-bd	canonical/nested-mixed	verify	848	1641	1.93×	+0	+0
rs:schubfach-vs-bd	canonical/numeric-boundary	verify	961	8740	9.09×	+0	+0
rs:schubfach-vs-bd	canonical/rfc-key-sorting	verify	1312	1321	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/small	verify	602	1013	1.68×	+0	+0
rs:schubfach-vs-bd	canonical/surrogate-pair	verify	283	297	1.05×	+0	+0
rs:schubfach-vs-bd	canonical/unicode	verify	1696	1631	1.04×	+0	+0
rs:schubfach-vs-bd	canonical/verify-whitespace	verify	364	978	2.69×	+0	+0
schubfach:go-vs-rs	array-2048	canonicalize	2541752	1185768	2.14×	-49134	-4939275
schubfach:go-vs-rs	array-256	canonicalize	305467	157030	1.95×	-6131	-604880
schubfach:go-vs-rs	canonical-minimal	canonicalize	351	190	1.84×	-8	-504
schubfach:go-vs-rs	control-escapes	canonicalize	1339	607	2.21×	-21	-2064
schubfach:go-vs-rs	deep-64	canonicalize	16230	9380	1.73×	-387	-26904
schubfach:go-vs-rs	escaped-key-order	canonicalize	708	298	2.37×	-14	-1168
schubfach:go-vs-rs	long-string	canonicalize	75831	17259	4.39×	-14	-35948

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
schubfach:go-vs-rs	nested-mixed	canonicalize	2184	843	2.59×	-45	-4128
schubfach:go-vs-rs	numeric-boundary	canonicalize	12693	976	13.00×	-38	-3208
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	3004	1294	2.32×	-47	-4256
schubfach:go-vs-rs	small	canonicalize	1251	583	2.15×	-22	-2152
schubfach:go-vs-rs	surrogate-pair	canonicalize	648	267	2.43×	-14	-1168
schubfach:go-vs-rs	unicode	canonicalize	1554	1676	1.08×	-19	-2019
schubfach:go-vs-rs	verify-whitespace	canonicalize	910	347	2.62×	-19	-1576
schubfach:go-vs-rs	canonical/array-2048	verify	2480516	1208080	2.05×	-49134	-4939271
schubfach:go-vs-rs	canonical/array-256	verify	316572	141609	2.24×	-6131	-604881
schubfach:go-vs-rs	canonical/canonical-minimal	verify	360	175	2.06×	-8	-504
schubfach:go-vs-rs	canonical/control-escapes	verify	1327	612	2.17×	-21	-2064
schubfach:go-vs-rs	canonical/deep-64	verify	16778	9154	1.83×	-387	-26904
schubfach:go-vs-rs	canonical/escaped-key-order	verify	690	286	2.41×	-14	-1160
schubfach:go-vs-rs	canonical/long-string	verify	77272	17384	4.44×	-14	-35948
schubfach:go-vs-rs	canonical/nested-mixed	verify	2219	848	2.62×	-45	-4128
schubfach:go-vs-rs	canonical/numeric-boundary	verify	12794	961	13.31×	-38	-3208
schubfach:go-vs-rs	canonical/rfc-key-sorting	verify	2857	1312	2.18×	-42	-4240
schubfach:go-vs-rs	canonical/small	verify	1223	602	2.03×	-22	-2152
schubfach:go-vs-rs	canonical/surrogate-pair	verify	658	283	2.33×	-14	-1160
schubfach:go-vs-rs	canonical/unicode	verify	1548	1696	1.10×	-19	-2003
schubfach:go-vs-rs	canonical/verify-whitespace	verify	934	364	2.57×	-19	-1576
bd:go-vs-rs	array-2048	canonicalize	4077596	2882258	1.41×	-57343	-5125917
bd:go-vs-rs	array-256	canonicalize	473478	321394	1.47×	-7151	-629857
bd:go-vs-rs	canonical-minimal	canonicalize	532	415	1.28×	-8	-504
bd:go-vs-rs	control-escapes	canonicalize	1258	607	2.07×	-21	-2064
bd:go-vs-rs	deep-64	canonicalize	16231	9185	1.77×	-387	-26904
bd:go-vs-rs	escaped-key-order	canonicalize	1129	746	1.51×	-16	-1209
bd:go-vs-rs	long-string	canonicalize	75639	17278	4.38×	-14	-35948
bd:go-vs-rs	nested-mixed	canonicalize	3089	1684	1.83×	-49	-4213
bd:go-vs-rs	numeric-boundary	canonicalize	18815	8975	2.10×	-52	-3914
bd:go-vs-rs	rfc-key-sorting	canonicalize	2973	1250	2.38×	-47	-4256
bd:go-vs-rs	small	canonicalize	1675	1000	1.67×	-24	-2202
bd:go-vs-rs	surrogate-pair	canonicalize	657	281	2.34×	-14	-1168
bd:go-vs-rs	unicode	canonicalize	1551	1964	1.27×	-19	-2019
bd:go-vs-rs	verify-whitespace	canonicalize	1569	980	1.60×	-23	-1658
bd:go-vs-rs	canonical/array-2048	verify	4028941	2906958	1.39×	-57342	-5125819
bd:go-vs-rs	canonical/array-256	verify	485479	309811	1.57×	-7152	-630092
bd:go-vs-rs	canonical/canonical-minimal	verify	548	413	1.33×	-8	-504
bd:go-vs-rs	canonical/control-escapes	verify	1271	606	2.10×	-21	-2064
bd:go-vs-rs	canonical/deep-64	verify	16830	9104	1.85×	-387	-26904
bd:go-vs-rs	canonical/escaped-key-order	verify	1130	724	1.56×	-16	-1201
bd:go-vs-rs	canonical/long-string	verify	77030	17325	4.45×	-14	-35948
bd:go-vs-rs	canonical/nested-mixed	verify	3107	1641	1.89×	-49	-4214
bd:go-vs-rs	canonical/numeric-boundary	verify	18861	8740	2.16×	-52	-3917
bd:go-vs-rs	canonical/rfc-key-sorting	verify	2843	1321	2.15×	-42	-4240
bd:go-vs-rs	canonical/small	verify	1656	1013	1.63×	-24	-2203

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
bd:go-vs-rs	canonical/surrogate-pair	verify	653	297	2.20×	-14	-1160
bd:go-vs-rs	canonical/unicode	verify	1547	1631	1.05×	-19	-2003
bd:go-vs-rs	canonical/verify-whitespace	verify	1588	978	1.62×	-23	-1658

B/op = bytes allocated per operation. allocs/op = heap allocations per operation.
Δ columns show Impl B − Impl A for each pair.

A.2 Full CLI Benchmarks

Table 5: CLI benchmark results (end-to-end, $n = 15$).

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	canonicalize	5.466	7.063	1.29×	schubfach
go:schubfach-vs-bd	array-2048	verify	5.623	7.128	1.27×	schubfach
go:schubfach-vs-bd	array-256	canonicalize	1.923	2.167	1.13×	schubfach
go:schubfach-vs-bd	array-256	verify	1.834	2.355	1.28×	schubfach
go:schubfach-vs-bd	canonical-minimal	canonicalize	1.510	1.565	1.04×	schubfach
go:schubfach-vs-bd	canonical-minimal	verify	1.386	1.389	1.00×	schubfach
go:schubfach-vs-bd	control-escapes	canonicalize	1.380	1.673	1.21×	schubfach
go:schubfach-vs-bd	control-escapes	verify	1.397	1.588	1.14×	schubfach
go:schubfach-vs-bd	deep-64	canonicalize	1.467	1.638	1.12×	schubfach
go:schubfach-vs-bd	deep-64	verify	1.484	1.798	1.21×	schubfach
go:schubfach-vs-bd	escaped-key-order	canonicalize	1.549	1.531	1.01×	json-canon
go:schubfach-vs-bd	escaped-key-order	verify	1.436	1.551	1.08×	schubfach
go:schubfach-vs-bd	long-string	canonicalize	1.409	1.692	1.20×	schubfach
go:schubfach-vs-bd	long-string	verify	1.659	1.718	1.04×	schubfach
go:schubfach-vs-bd	nested-mixed	canonicalize	1.511	1.497	1.01×	json-canon
go:schubfach-vs-bd	nested-mixed	verify	1.453	1.676	1.15×	schubfach
go:schubfach-vs-bd	numeric-boundary	canonicalize	1.537	1.706	1.11×	schubfach
go:schubfach-vs-bd	numeric-boundary	verify	1.432	1.599	1.12×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	1.561	1.608	1.03×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	verify	1.493	1.698	1.14×	schubfach
go:schubfach-vs-bd	small	canonicalize	1.450	1.449	1.00×	json-canon
go:schubfach-vs-bd	small	verify	1.402	1.702	1.21×	schubfach
go:schubfach-vs-bd	surrogate-pair	canonicalize	1.510	1.524	1.01×	schubfach
go:schubfach-vs-bd	surrogate-pair	verify	1.448	1.625	1.12×	schubfach
go:schubfach-vs-bd	unicode	canonicalize	1.597	1.470	1.09×	json-canon
go:schubfach-vs-bd	unicode	verify	1.400	1.663	1.19×	schubfach
go:schubfach-vs-bd	verify-whitespace	canonicalize	1.455	1.614	1.11×	schubfach
go:schubfach-vs-bd	verify-whitespace	verify	1.448	1.570	1.08×	schubfach
rs:schubfach-vs-bd	array-2048	canonicalize	2.606	4.584	1.76×	schubfach-rs
rs:schubfach-vs-bd	array-2048	verify	2.419	4.146	1.71×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
rs:schubfach-vs-bd	array-256	canonicalize	0.828	1.152	1.39×	schubfach-rs
rs:schubfach-vs-bd	array-256	verify	0.821	1.217	1.48×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	canonicalize	0.546	0.582	1.07×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	verify	0.557	0.581	1.04×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	canonicalize	0.531	0.594	1.12×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	verify	0.537	0.617	1.15×	schubfach-rs
rs:schubfach-vs-bd	deep-64	canonicalize	0.564	0.647	1.15×	schubfach-rs
rs:schubfach-vs-bd	deep-64	verify	0.561	0.609	1.08×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	canonicalize	0.573	0.712	1.24×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	verify	0.553	0.645	1.17×	schubfach-rs
rs:schubfach-vs-bd	long-string	canonicalize	0.605	0.636	1.05×	schubfach-rs
rs:schubfach-vs-bd	long-string	verify	0.612	0.647	1.06×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	canonicalize	0.555	0.644	1.16×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	verify	0.546	0.639	1.17×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	canonicalize	0.543	0.738	1.36×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	verify	0.545	0.715	1.31×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	0.535	0.588	1.10×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	verify	0.534	0.601	1.12×	schubfach-rs
rs:schubfach-vs-bd	small	canonicalize	0.530	0.642	1.21×	schubfach-rs
rs:schubfach-vs-bd	small	verify	0.546	0.708	1.30×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	canonicalize	0.536	0.603	1.12×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	verify	0.533	0.578	1.08×	schubfach-rs
rs:schubfach-vs-bd	unicode	canonicalize	0.534	0.612	1.15×	schubfach-rs
rs:schubfach-vs-bd	unicode	verify	0.546	0.629	1.15×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	canonicalize	0.559	0.691	1.24×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	verify	0.554	0.688	1.24×	schubfach-rs
schubfach:go-vs-rs	array-2048	canonicalize	5.466	2.606	2.10×	schubfach-rs
schubfach:go-vs-rs	array-2048	verify	5.623	2.419	2.32×	schubfach-rs
schubfach:go-vs-rs	array-256	canonicalize	1.923	0.828	2.32×	schubfach-rs
schubfach:go-vs-rs	array-256	verify	1.834	0.821	2.23×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	canonicalize	1.510	0.546	2.77×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	verify	1.386	0.557	2.49×	schubfach-rs
schubfach:go-vs-rs	control-escapes	canonicalize	1.380	0.531	2.60×	schubfach-rs
schubfach:go-vs-rs	control-escapes	verify	1.397	0.537	2.60×	schubfach-rs
schubfach:go-vs-rs	deep-64	canonicalize	1.467	0.564	2.60×	schubfach-rs
schubfach:go-vs-rs	deep-64	verify	1.484	0.561	2.64×	schubfach-rs
schubfach:go-vs-rs	escaped-key-order	canonicalize	1.549	0.573	2.70×	schubfach-rs
schubfach:go-vs-rs	escaped-key-order	verify	1.436	0.553	2.60×	schubfach-rs
schubfach:go-vs-rs	long-string	canonicalize	1.409	0.605	2.33×	schubfach-rs
schubfach:go-vs-rs	long-string	verify	1.659	0.612	2.71×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	canonicalize	1.511	0.555	2.72×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	verify	1.453	0.546	2.66×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	canonicalize	1.537	0.543	2.83×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	verify	1.432	0.545	2.63×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	1.561	0.535	2.92×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	verify	1.493	0.534	2.79×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
schubfach:go-vs-rs	small	canonicalize	1.450	0.530	2.73×	schubfach-rs
schubfach:go-vs-rs	small	verify	1.402	0.546	2.56×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	canonicalize	1.510	0.536	2.82×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	verify	1.448	0.533	2.72×	schubfach-rs
schubfach:go-vs-rs	unicode	canonicalize	1.597	0.534	2.99×	schubfach-rs
schubfach:go-vs-rs	unicode	verify	1.400	0.546	2.56×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	canonicalize	1.455	0.559	2.61×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	verify	1.448	0.554	2.62×	schubfach-rs
bd:go-vs-rs	array-2048	canonicalize	7.063	4.584	1.54×	json-canon-rs
bd:go-vs-rs	array-2048	verify	7.128	4.146	1.72×	json-canon-rs
bd:go-vs-rs	array-256	canonicalize	2.167	1.152	1.88×	json-canon-rs
bd:go-vs-rs	array-256	verify	2.355	1.217	1.93×	json-canon-rs
bd:go-vs-rs	canonical-minimal	canonicalize	1.565	0.582	2.69×	json-canon-rs
bd:go-vs-rs	canonical-minimal	verify	1.389	0.581	2.39×	json-canon-rs
bd:go-vs-rs	control-escapes	canonicalize	1.673	0.594	2.82×	json-canon-rs
bd:go-vs-rs	control-escapes	verify	1.588	0.617	2.57×	json-canon-rs
bd:go-vs-rs	deep-64	canonicalize	1.638	0.647	2.53×	json-canon-rs
bd:go-vs-rs	deep-64	verify	1.798	0.609	2.95×	json-canon-rs
bd:go-vs-rs	escaped-key-order	canonicalize	1.531	0.712	2.15×	json-canon-rs
bd:go-vs-rs	escaped-key-order	verify	1.551	0.645	2.40×	json-canon-rs
bd:go-vs-rs	long-string	canonicalize	1.692	0.636	2.66×	json-canon-rs
bd:go-vs-rs	long-string	verify	1.718	0.647	2.66×	json-canon-rs
bd:go-vs-rs	nested-mixed	canonicalize	1.497	0.644	2.32×	json-canon-rs
bd:go-vs-rs	nested-mixed	verify	1.676	0.639	2.62×	json-canon-rs
bd:go-vs-rs	numeric-boundary	canonicalize	1.706	0.738	2.31×	json-canon-rs
bd:go-vs-rs	numeric-boundary	verify	1.599	0.715	2.23×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	canonicalize	1.608	0.588	2.73×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	verify	1.698	0.601	2.83×	json-canon-rs
bd:go-vs-rs	small	canonicalize	1.449	0.642	2.26×	json-canon-rs
bd:go-vs-rs	small	verify	1.702	0.708	2.40×	json-canon-rs
bd:go-vs-rs	surrogate-pair	canonicalize	1.524	0.603	2.53×	json-canon-rs
bd:go-vs-rs	surrogate-pair	verify	1.625	0.578	2.81×	json-canon-rs
bd:go-vs-rs	unicode	canonicalize	1.470	0.612	2.40×	json-canon-rs
bd:go-vs-rs	unicode	verify	1.663	0.629	2.65×	json-canon-rs
bd:go-vs-rs	verify-whitespace	canonicalize	1.614	0.691	2.34×	json-canon-rs
bd:go-vs-rs	verify-whitespace	verify	1.570	0.688	2.28×	json-canon-rs

Speedup = slower/faster mean wall time. $n = 15$ repetitions, 3 warmup runs discarded.

A.3 Full Statistical Comparisons

Table 6: Statistical comparisons with Benjamini-Hochberg corrected p-values.

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
go:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach	json-canon	schubfach*	1.29×	[1.210, 1.384]	0.0002	0.0003	-2.67	
rs:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	1.76×	[1.578, 1.992]	0.0002	0.0003	-3.22	
schubfach:go-vs-rs	e2e	canonicalize	array-2048	schubfach	schubfach-rs	schubfach-rs*	2.10×	[1.910, 2.329]	0.0002	0.0003	5.37	
bd:go-vs-rs	e2e	canonicalize	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.54×	[1.394, 1.667]	0.0002	0.0003	3.69	
go:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach	json-canon	schubfach	1.13×	[0.991, 1.282]	0.0912	0.1072	-0.63	
rs:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	1.39×	[1.262, 1.515]	0.0002	0.0003	-2.33	
schubfach:go-vs-rs	e2e	canonicalize	array-256	schubfach	schubfach-rs	schubfach-rs*	2.32×	[2.079, 2.589]	0.0002	0.0003	3.87	
bd:go-vs-rs	e2e	canonicalize	array-256	json-canon	json-canon-rs	json-canon-rs*	1.88×	[1.693, 2.122]	0.0002	0.0003	3.41	
go:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	schubfach	json-canon	schubfach	1.04×	[0.917, 1.163]	0.5691	0.6046	-0.21	
rs:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs	1.07×	[0.987, 1.159]	0.1492	0.1698	-0.55	
schubfach:go-vs-rs	e2e	canonicalize	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	2.77×	[2.517, 3.003]	0.0002	0.0003	6.06	
bd:go-vs-rs	e2e	canonicalize	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	2.69×	[2.394, 2.998]	0.0002	0.0003	4.52	
go:schubfach-vs-bd	e2e	canonicalize	control-escapes	schubfach	json-canon	schubfach*	1.21×	[1.083, 1.369]	0.0046	0.0059	-1.17	
rs:schubfach-vs-bd	e2e	canonicalize	control-escapes	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.030, 1.199]	0.0110	0.0139	-1.00	
schubfach:go-vs-rs	e2e	canonicalize	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.60×	[2.316, 2.883]	0.0002	0.0003	4.13	
bd:go-vs-rs	e2e	canonicalize	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.82×	[2.598, 3.076]	0.0002	0.0003	6.91	
go:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach	json-canon	schubfach	1.12×	[0.997, 1.259]	0.0864	0.1021	-0.65	
rs:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.067, 1.258]	0.0012	0.0016	-1.21	
schubfach:go-vs-rs	e2e	canonicalize	deep-64	schubfach	schubfach-rs	schubfach-rs*	2.60×	[2.339, 2.906]	0.0002	0.0003	4.70	
bd:go-vs-rs	e2e	canonicalize	deep-64	json-canon	json-canon-rs	json-canon-rs*	2.53×	[2.291, 2.759]	0.0002	0.0003	5.07	
go:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach	json-canon	json-canon	1.01×	[0.899, 1.138]	0.8612	0.8863	0.07	
rs:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	1.24×	[1.178, 1.302]	0.0002	0.0003	-2.99	
schubfach:go-vs-rs	e2e	canonicalize	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.70×	[2.423, 2.893]	0.0002	0.0003	5.73	
bd:go-vs-rs	e2e	canonicalize	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	2.15×	[1.939, 2.354]	0.0002	0.0003	3.98	
go:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach	json-canon	schubfach*	1.20×	[1.031, 1.394]	0.0292	0.0359	-0.84	
rs:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach-rs	json-canon-rs	schubfach-rs	1.05×	[0.958, 1.158]	0.3217	0.3516	-0.37	
schubfach:go-vs-rs	e2e	canonicalize	long-string	schubfach	schubfach-rs	schubfach-rs*	2.33×	[2.067, 2.658]	0.0002	0.0003	3.53	
bd:go-vs-rs	e2e	canonicalize	long-string	json-canon	json-canon-rs	json-canon-rs*	2.66×	[2.333, 3.005]	0.0002	0.0003	4.01	
go:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach	json-canon	json-canon	1.01×	[0.894, 1.147]	0.8732	0.8943	0.05	
rs:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.16×	[1.057, 1.253]	0.0034	0.0044	-1.17	
schubfach:go-vs-rs	e2e	canonicalize	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.72×	[2.509, 2.934]	0.0002	0.0003	6.42	
bd:go-vs-rs	e2e	canonicalize	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	2.32×	[2.028, 2.634]	0.0002	0.0003	3.53	
go:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach	json-canon	schubfach	1.11×	[0.985, 1.236]	0.0934	0.1093	-0.63	
rs:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	1.36×	[1.295, 1.427]	0.0002	0.0003	-4.45	
schubfach:go-vs-rs	e2e	canonicalize	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	2.83×	[2.599, 3.044]	0.0002	0.0003	6.30	
bd:go-vs-rs	e2e	canonicalize	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.31×	[2.071, 2.510]	0.0002	0.0003	4.34	
go:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	schubfach	json-canon	schubfach	1.03×	[0.924, 1.142]	0.5871	0.6197	-0.20	
rs:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	schubfach-rs	json-canon-rs	schubfach-rs*	1.10×	[1.021, 1.180]	0.0214	0.0266	-0.89	
schubfach:go-vs-rs	e2e	canonicalize	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.92×	[2.666, 3.134]	0.0002	0.0003	6.61	
bd:go-vs-rs	e2e	canonicalize	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.73×	[2.463, 2.976]	0.0002	0.0003	5.56	
go:schubfach-vs-bd	e2e	canonicalize	small	schubfach	json-canon	json-canon	1.00×	[0.877, 1.129]	0.9942	0.9942	0.00	
rs:schubfach-vs-bd	e2e	canonicalize	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.21×	[1.098, 1.326]	0.0014	0.0018	-1.31	
schubfach:go-vs-rs	e2e	canonicalize	small	schubfach	schubfach-rs	schubfach-rs*	2.73×	[2.449, 3.015]	0.0002	0.0003	4.88	

C

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
bd:go-vs-rs	e2e	canonicalize	small	json-canon	json-canon-rs	json-canon-rs*	2.26×	[2.003, 2.557]	0.0002	0.0003	3.90	
go:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach	json-canon	schubfach	1.01×	[0.904, 1.133]	0.8762	0.8943	-0.06	
rs:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.034, 1.233]	0.0210	0.0262	-0.91	
schubfach:go-vs-rs	e2e	canonicalize	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.82×	[2.549, 3.094]	0.0002	0.0003	5.78	
bd:go-vs-rs	e2e	canonicalize	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.53×	[2.261, 2.816]	0.0002	0.0003	4.67	
go:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach	json-canon	json-canon	1.09×	[0.964, 1.213]	0.1568	0.1776	0.52	
rs:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.059, 1.245]	0.0034	0.0044	-1.15	
schubfach:go-vs-rs	e2e	canonicalize	unicode	schubfach	schubfach-rs	schubfach-rs*	2.99×	[2.743, 3.255]	0.0002	0.0003	6.95	
bd:go-vs-rs	e2e	canonicalize	unicode	json-canon	json-canon-rs	json-canon-rs*	2.40×	[2.166, 2.692]	0.0002	0.0003	4.28	
go:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach	json-canon	schubfach	1.11×	[0.961, 1.267]	0.1672	0.1885	-0.51	
rs:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	1.24×	[1.132, 1.334]	0.0004	0.0005	-1.67	
schubfach:go-vs-rs	e2e	canonicalize	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.61×	[2.345, 2.869]	0.0002	0.0003	4.70	
bd:go-vs-rs	e2e	canonicalize	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	2.34×	[2.051, 2.662]	0.0002	0.0003	3.55	
go:schubfach-vs-bd	e2e	verify	array-2048	schubfach	json-canon	schubfach*	1.27×	[1.205, 1.348]	0.0002	0.0003	-2.79	
rs:schubfach-vs-bd	e2e	verify	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	1.71×	[1.549, 1.917]	0.0002	0.0003	-3.52	
schubfach:go-vs-rs	e2e	verify	array-2048	schubfach	schubfach-rs	schubfach-rs*	2.32×	[2.096, 2.544]	0.0002	0.0003	7.37	
bd:go-vs-rs	e2e	verify	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.72×	[1.563, 1.826]	0.0002	0.0003	5.10	
go:schubfach-vs-bd	e2e	verify	array-256	schubfach	json-canon	schubfach*	1.28×	[1.130, 1.454]	0.0018	0.0024	-1.39	
rs:schubfach-vs-bd	e2e	verify	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	1.48×	[1.359, 1.622]	0.0002	0.0003	-2.96	
schubfach:go-vs-rs	e2e	verify	array-256	schubfach	schubfach-rs	schubfach-rs*	2.23×	[2.011, 2.503]	0.0002	0.0003	4.16	
bd:go-vs-rs	e2e	verify	array-256	json-canon	json-canon-rs	json-canon-rs*	1.93×	[1.739, 2.166]	0.0002	0.0003	3.60	
go:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach	json-canon	schubfach	1.00×	[0.882, 1.141]	0.9748	0.9868	-0.01	
rs:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs	1.04×	[0.966, 1.116]	0.2881	0.3163	-0.40	
schubfach:go-vs-rs	e2e	verify	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	2.49×	[2.269, 2.732]	0.0002	0.0003	4.69	
bd:go-vs-rs	e2e	verify	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	2.39×	[2.141, 2.694]	0.0002	0.0003	4.08	
go:schubfach-vs-bd	e2e	verify	control-escapes	schubfach	json-canon	schubfach*	1.14×	[1.027, 1.259]	0.0310	0.0379	-0.84	
rs:schubfach-vs-bd	e2e	verify	control-escapes	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.087, 1.235]	0.0004	0.0005	-1.58	
schubfach:go-vs-rs	e2e	verify	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.60×	[2.386, 2.843]	0.0002	0.0003	5.92	
bd:go-vs-rs	e2e	verify	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.57×	[2.353, 2.780]	0.0002	0.0003	5.34	
go:schubfach-vs-bd	e2e	verify	deep-64	schubfach	json-canon	schubfach*	1.21×	[1.075, 1.379]	0.0074	0.0094	-1.10	
rs:schubfach-vs-bd	e2e	verify	deep-64	schubfach-rs	json-canon-rs	schubfach-rs	1.08×	[0.983, 1.198]	0.1254	0.1447	-0.58	
schubfach:go-vs-rs	e2e	verify	deep-64	schubfach	schubfach-rs	schubfach-rs*	2.64×	[2.307, 3.012]	0.0002	0.0003	3.63	
bd:go-vs-rs	e2e	verify	deep-64	json-canon	json-canon-rs	json-canon-rs*	2.95×	[2.715, 3.256]	0.0002	0.0003	7.76	
go:schubfach-vs-bd	e2e	verify	escaped-key-order	schubfach	json-canon	schubfach	1.08×	[0.955, 1.229]	0.2691	0.2995	-0.42	
rs:schubfach-vs-bd	e2e	verify	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	1.17×	[1.090, 1.250]	0.0006	0.0008	-1.54	
schubfach:go-vs-rs	e2e	verify	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.60×	[2.323, 2.877]	0.0002	0.0003	4.40	
bd:go-vs-rs	e2e	verify	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	2.40×	[2.184, 2.652]	0.0002	0.0003	4.64	
go:schubfach-vs-bd	e2e	verify	long-string	schubfach	json-canon	schubfach	1.04×	[0.895, 1.196]	0.6385	0.6711	-0.17	
rs:schubfach-vs-bd	e2e	verify	long-string	schubfach-rs	json-canon-rs	schubfach-rs	1.06×	[0.987, 1.126]	0.1222	0.1423	-0.59	
schubfach:go-vs-rs	e2e	verify	long-string	schubfach	schubfach-rs	schubfach-rs*	2.71×	[2.407, 2.986]	0.0002	0.0003	4.32	
bd:go-vs-rs	e2e	verify	long-string	json-canon	json-canon-rs	json-canon-rs*	2.66×	[2.361, 2.960]	0.0002	0.0003	4.10	
go:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach	json-canon	schubfach*	1.15×	[1.016, 1.302]	0.0314	0.0382	-0.82	
rs:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.17×	[1.060, 1.274]	0.0054	0.0069	-1.12	
schubfach:go-vs-rs	e2e	verify	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.66×	[2.400, 2.916]	0.0002	0.0003	4.65	
bd:go-vs-rs	e2e	verify	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	2.62×	[2.335, 2.951]	0.0002	0.0003	5.01	
go:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach	json-canon	schubfach	1.12×	[0.988, 1.280]	0.1252	0.1447	-0.58	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
rs:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	1.31×	[1.225, 1.383]	0.0002	0.0003	-3.04	
schubfach:go-vs-rs	e2e	verify	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	2.63×	[2.326, 2.894]	0.0002	0.0003	4.14	
bd:go-vs-rs	e2e	verify	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.23×	[2.028, 2.460]	0.0002	0.0003	4.42	
go:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach	json-canon	schubfach*	1.14×	[1.016, 1.267]	0.0364	0.0441	-0.80	
rs:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.023, 1.215]	0.0206	0.0258	-0.93	
schubfach:go-vs-rs	e2e	verify	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.79×	[2.504, 3.060]	0.0002	0.0003	5.37	
bd:go-vs-rs	e2e	verify	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.83×	[2.548, 3.139]	0.0002	0.0003	5.63	
go:schubfach-vs-bd	e2e	verify	small	schubfach	json-canon	schubfach*	1.21×	[1.080, 1.357]	0.0020	0.0026	-1.23	
rs:schubfach-vs-bd	e2e	verify	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.30×	[1.220, 1.385]	0.0002	0.0003	-2.74	
schubfach:go-vs-rs	e2e	verify	small	schubfach	schubfach-rs	schubfach-rs*	2.56×	[2.289, 2.846]	0.0002	0.0003	4.31	
bd:go-vs-rs	e2e	verify	small	json-canon	json-canon-rs	json-canon-rs*	2.40×	[2.202, 2.577]	0.0002	0.0003	6.42	
go:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach	json-canon	schubfach	1.12×	[0.993, 1.266]	0.0852	0.1012	-0.65	
rs:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs	1.08×	[0.977, 1.194]	0.1422	0.1633	-0.54	
schubfach:go-vs-rs	e2e	verify	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.72×	[2.422, 3.001]	0.0002	0.0003	4.88	
bd:go-vs-rs	e2e	verify	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.81×	[2.481, 3.161]	0.0002	0.0003	4.94	
go:schubfach-vs-bd	e2e	verify	unicode	schubfach	json-canon	schubfach*	1.19×	[1.022, 1.368]	0.0286	0.0353	-0.84	
rs:schubfach-vs-bd	e2e	verify	unicode	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.084, 1.248]	0.0006	0.0008	-1.42	
schubfach:go-vs-rs	e2e	verify	unicode	schubfach	schubfach-rs	schubfach-rs*	2.56×	[2.257, 2.894]	0.0002	0.0003	3.60	
bd:go-vs-rs	e2e	verify	unicode	json-canon	json-canon-rs	json-canon-rs*	2.65×	[2.376, 2.880]	0.0002	0.0003	4.89	
go:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach	json-canon	schubfach	1.08×	[0.946, 1.263]	0.2821	0.3125	-0.39	
rs:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	1.24×	[1.150, 1.329]	0.0002	0.0003	-2.00	
schubfach:go-vs-rs	e2e	verify	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.62×	[2.291, 2.938]	0.0002	0.0003	3.62	
bd:go-vs-rs	e2e	verify	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	2.28×	[2.062, 2.524]	0.0002	0.0003	4.45	
go:schubfach-vs-bd	api	canonicalize	array-2048	schubfach	json-canon	schubfach*	1.60×	[1.596, 1.612]	0.0002	0.0003	-70.44	
rs:schubfach-vs-bd	api	canonicalize	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	2.43×	[2.401, 2.458]	0.0002	0.0003	-63.57	
schubfach:go-vs-rs	api	canonicalize	array-2048	schubfach	schubfach-rs	schubfach-rs*	2.14×	[2.122, 2.164]	0.0002	0.0003	80.86	
bd:go-vs-rs	api	canonicalize	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.41×	[1.405, 1.427]	0.0002	0.0003	39.70	
go:schubfach-vs-bd	api	canonicalize	array-256	schubfach	json-canon	schubfach*	1.55×	[1.540, 1.563]	0.0002	0.0003	-44.46	
rs:schubfach-vs-bd	api	canonicalize	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	2.05×	[2.005, 2.085]	0.0002	0.0003	-26.64	
schubfach:go-vs-rs	api	canonicalize	array-256	schubfach	schubfach-rs	schubfach-rs*	1.95×	[1.915, 1.968]	0.0002	0.0003	51.70	
bd:go-vs-rs	api	canonicalize	array-256	json-canon	json-canon-rs	json-canon-rs*	1.47×	[1.449, 1.496]	0.0002	0.0003	22.90	
go:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach	json-canon	schubfach*	1.51×	[1.506, 1.522]	0.0002	0.0003	-60.86	
rs:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	2.18×	[1.981, 2.314]	0.0002	0.0003	-11.03	
schubfach:go-vs-rs	api	canonicalize	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	1.84×	[1.682, 1.953]	0.0002	0.0003	9.56	
bd:go-vs-rs	api	canonicalize	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	1.28×	[1.248, 1.309]	0.0002	0.0003	9.84	
go:schubfach-vs-bd	api	canonicalize	control-escapes	schubfach	json-canon	json-canon*	1.06×	[1.059, 1.074]	0.0002	0.0003	8.01	
rs:schubfach-vs-bd	api	canonicalize	control-escapes	schubfach-rs	json-canon-rs	schubfach-rs	1.00×	[0.968, 1.035]	0.9892	0.9942	-0.00	
schubfach:go-vs-rs	api	canonicalize	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.21×	[2.166, 2.254]	0.0002	0.0003	46.14	
bd:go-vs-rs	api	canonicalize	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.07×	[2.016, 2.131]	0.0002	0.0003	30.61	
go:schubfach-vs-bd	api	canonicalize	deep-64	schubfach	json-canon	schubfach	1.00×	[0.995, 1.005]	0.9936	0.9942	-0.00	
rs:schubfach-vs-bd	api	canonicalize	deep-64	schubfach-rs	json-canon-rs	json-canon-rs	1.02×	[0.987, 1.135]	0.9502	0.9659	0.30	
schubfach:go-vs-rs	api	canonicalize	deep-64	schubfach	schubfach-rs	schubfach-rs*	1.73×	[1.574, 1.788]	0.0002	0.0003	10.64	
bd:go-vs-rs	api	canonicalize	deep-64	json-canon	json-canon-rs	json-canon-rs*	1.77×	[1.753, 1.787]	0.0002	0.0003	59.52	
go:schubfach-vs-bd	api	canonicalize	deep	schubfach	json-canon	schubfach	1.00×	[0.994, 1.007]	0.8556	0.8843	-0.08	
go:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach	json-canon	schubfach*	1.60×	[1.590, 1.605]	0.0002	0.0003	-78.83	
rs:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	2.50×	[2.429, 2.575]	0.0002	0.0003	-22.95	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
schubfach:go-vs-rs	api	canonicalize	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.37×	[2.324, 2.424]	0.0002	0.0003	52.49	
bd:go-vs-rs	api	canonicalize	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	1.51×	[1.486, 1.548]	0.0002	0.0003	20.47	
go:schubfach-vs-bd	api	canonicalize	large	schubfach	json-canon	schubfach*	1.73×	[1.717, 1.733]	0.0002	0.0003	-88.68	
go:schubfach-vs-bd	api	canonicalize	long-string	schubfach	json-canon	json-canon	1.00×	[0.992, 1.016]	0.6959	0.7252	0.18	
rs:schubfach-vs-bd	api	canonicalize	long-string	schubfach-rs	json-canon-rs	schubfach-rs	1.00×	[0.999, 1.003]	0.2845	0.3138	-0.50	
schubfach:go-vs-rs	api	canonicalize	long-string	schubfach	schubfach-rs	schubfach-rs*	4.39×	[4.364, 4.429]	0.0002	0.0003	90.07	
bd:go-vs-rs	api	canonicalize	long-string	json-canon	json-canon-rs	json-canon-rs*	4.38×	[4.329, 4.414]	0.0002	0.0003	67.27	
go:schubfach-vs-bd	api	canonicalize	medium	schubfach	json-canon	schubfach*	1.71×	[1.700, 1.720]	0.0002	0.0003	-70.44	
go:schubfach-vs-bd	api	canonicalize	mixed-prod	schubfach	json-canon	schubfach*	1.32×	[1.313, 1.335]	0.0002	0.0003	-28.26	
go:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach	json-canon	schubfach*	1.41×	[1.406, 1.423]	0.0002	0.0003	-47.44	
rs:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	2.00×	[1.918, 2.091]	0.0002	0.0003	-14.28	
schubfach:go-vs-rs	api	canonicalize	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.59×	[2.507, 2.690]	0.0002	0.0003	35.46	
bd:go-vs-rs	api	canonicalize	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	1.83×	[1.780, 1.867]	0.0002	0.0003	28.63	
go:schubfach-vs-bd	api	canonicalize	number-heavy	schubfach	json-canon	schubfach*	1.55×	[1.539, 1.564]	0.0002	0.0003	-43.91	
go:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach	json-canon	schubfach*	1.48×	[1.470, 1.496]	0.0002	0.0003	-34.77	
rs:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	9.19×	[8.815, 9.382]	0.0002	0.0003	-57.11	
schubfach:go-vs-rs	api	canonicalize	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	13.00×	[12.499, 13.235]	0.0002	0.0003	150.36	
bd:go-vs-rs	api	canonicalize	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.10×	[2.067, 2.127]	0.0002	0.0003	46.62	
go:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	schubfach	json-canon	json-canon*	1.01×	[1.006, 1.014]	0.0002	0.0003	2.35	
rs:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	schubfach-rs	json-canon-rs	json-canon-rs	1.04×	[1.001, 1.072]	0.0768	0.0916	0.85	
schubfach:go-vs-rs	api	canonicalize	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.32×	[2.290, 2.353]	0.0002	0.0003	74.22	
bd:go-vs-rs	api	canonicalize	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.38×	[2.303, 2.455]	0.0002	0.0003	35.37	
go:schubfach-vs-bd	api	canonicalize	small	schubfach	json-canon	schubfach*	1.34×	[1.315, 1.351]	0.0002	0.0003	-20.55	
rs:schubfach-vs-bd	api	canonicalize	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.71×	[1.655, 1.780]	0.0002	0.0003	-12.75	
schubfach:go-vs-rs	api	canonicalize	small	schubfach	schubfach-rs	schubfach-rs*	2.15×	[2.080, 2.222]	0.0002	0.0003	24.16	
bd:go-vs-rs	api	canonicalize	small	json-canon	json-canon-rs	json-canon-rs*	1.67×	[1.642, 1.713]	0.0002	0.0003	25.00	
go:schubfach-vs-bd	api	canonicalize	surrogate-pair	schubfach	json-canon	schubfach	1.01×	[0.997, 1.038]	0.1984	0.2217	-0.60	
rs:schubfach-vs-bd	api	canonicalize	surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs	1.05×	[0.993, 1.119]	0.1452	0.1660	-0.68	
schubfach:go-vs-rs	api	canonicalize	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.42×	[2.315, 2.496]	0.0002	0.0003	30.62	
bd:go-vs-rs	api	canonicalize	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.34×	[2.204, 2.450]	0.0002	0.0003	17.04	
go:schubfach-vs-bd	api	canonicalize	unicode	schubfach	json-canon	json-canon	1.00×	[0.998, 1.006]	0.3375	0.3673	0.44	
rs:schubfach-vs-bd	api	canonicalize	unicode	schubfach-rs	json-canon-rs	schubfach-rs	1.17×	[1.049, 1.358]	0.0416	0.0501	-0.98	
schubfach:go-vs-rs	api	canonicalize	unicode	schubfach	schubfach-rs	schubfach*	1.08×	[1.065, 1.103]	0.0002	0.0003	-3.65	
bd:go-vs-rs	api	canonicalize	unicode	json-canon	json-canon-rs	json-canon*	1.27×	[1.136, 1.457]	0.0002	0.0003	-1.41	
go:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach	json-canon	schubfach*	1.72×	[1.715, 1.730]	0.0002	0.0003	-112.08	
rs:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	2.83×	[2.721, 2.933]	0.0002	0.0003	-21.44	
schubfach:go-vs-rs	api	canonicalize	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.62×	[2.538, 2.691]	0.0002	0.0003	44.60	
bd:go-vs-rs	api	canonicalize	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	1.60×	[1.553, 1.631]	0.0002	0.0003	21.50	
go:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach	json-canon	schubfach*	1.62×	[1.617, 1.631]	0.0002	0.0003	-82.30	
rs:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	2.41×	[2.376, 2.451]	0.0002	0.0003	-41.10	
schubfach:go-vs-rs	api	verify	canonical/array-2048	schubfach	schubfach-rs	schubfach-rs*	2.05×	[2.030, 2.075]	0.0002	0.0003	72.73	
bd:go-vs-rs	api	verify	canonical/array-2048	json-canon	json-canon-rs	json-canon-rs*	1.39×	[1.359, 1.397]	0.0002	0.0003	26.77	
go:schubfach-vs-bd	api	verify	canonical/array-256	schubfach	json-canon	schubfach*	1.53×	[1.526, 1.541]	0.0002	0.0003	-73.56	
rs:schubfach-vs-bd	api	verify	canonical/array-256	schubfach-rs	json-canon-rs	schubfach-rs*	2.19×	[1.947, 2.320]	0.0002	0.0003	-12.47	
schubfach:go-vs-rs	api	verify	canonical/array-256	schubfach	schubfach-rs	schubfach-rs*	2.24×	[1.991, 2.367]	0.0002	0.0003	13.04	
bd:go-vs-rs	api	verify	canonical/array-256	json-canon	json-canon-rs	json-canon-rs*	1.57×	[1.556, 1.576]	0.0002	0.0003	66.13	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
go:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach	json-canon	schubfach*	1.52×	[1.514, 1.533]	0.0002	0.0003	-47.82	
rs:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	2.36×	[2.236, 2.476]	0.0002	0.0003	-14.56	
schubfach:go-vs-rs	api	verify	canonical/canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	2.06×	[1.947, 2.123]	0.0002	0.0003	21.10	
bd:go-vs-rs	api	verify	canonical/canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	1.32×	[1.278, 1.355]	0.0002	0.0003	9.33	
go:schubfach-vs-bd	api	verify	canonical/control-escapes	schubfach	json-canon	json-canon*	1.04×	[1.040, 1.049]	0.0002	0.0003	8.52	
rs:schubfach-vs-bd	api	verify	canonical/control-escapes	schubfach-rs	json-canon-rs	json-canon-rs	1.01×	[0.983, 1.041]	0.5247	0.5635	0.29	
schubfach:go-vs-rs	api	verify	canonical/control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.17×	[2.120, 2.206]	0.0002	0.0003	50.21	
bd:go-vs-rs	api	verify	canonical/control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.10×	[2.054, 2.143]	0.0002	0.0003	40.69	
go:schubfach-vs-bd	api	verify	canonical/deep-64	schubfach	json-canon	schubfach	1.00×	[0.999, 1.007]	0.1822	0.2045	-0.63	
rs:schubfach-vs-bd	api	verify	canonical/deep-64	schubfach-rs	json-canon-rs	json-canon-rs	1.01×	[0.997, 1.019]	0.3689	0.3979	0.43	
schubfach:go-vs-rs	api	verify	canonical/deep-64	schubfach	schubfach-rs	schubfach-rs*	1.83×	[1.813, 1.847]	0.0004	0.0005	65.66	
bd:go-vs-rs	api	verify	canonical/deep-64	json-canon	json-canon-rs	json-canon-rs*	1.85×	[1.839, 1.865]	0.0002	0.0003	93.85	
go:schubfach-vs-bd	api	verify	canonical/deep	schubfach	json-canon	schubfach*	1.01×	[1.007, 1.018]	0.0004	0.0005	-1.69	
go:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach	json-canon	schubfach*	1.64×	[1.630, 1.648]	0.0002	0.0003	-71.81	
rs:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	2.53×	[2.429, 2.666]	0.0002	0.0003	-14.06	
schubfach:go-vs-rs	api	verify	canonical/escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.41×	[2.315, 2.472]	0.0002	0.0003	36.29	
bd:go-vs-rs	api	verify	canonical/escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	1.56×	[1.480, 1.596]	0.0002	0.0003	13.62	
go:schubfach-vs-bd	api	verify	canonical/large	schubfach	json-canon	schubfach*	1.74×	[1.727, 1.743]	0.0002	0.0003	-89.43	
go:schubfach-vs-bd	api	verify	canonical/long-string	schubfach	json-canon	json-canon	1.00×	[0.993, 1.012]	0.5451	0.5828	0.27	
rs:schubfach-vs-bd	api	verify	canonical/long-string	schubfach-rs	json-canon-rs	json-canon-rs	1.00×	[0.993, 1.016]	0.5703	0.6046	0.26	
schubfach:go-vs-rs	api	verify	canonical/long-string	schubfach	schubfach-rs	schubfach-rs*	4.44×	[4.407, 4.498]	0.0004	0.0005	87.44	
bd:go-vs-rs	api	verify	canonical/long-string	json-canon	json-canon-rs	json-canon-rs*	4.45×	[4.408, 4.505]	0.0002	0.0003	97.80	
go:schubfach-vs-bd	api	verify	canonical/medium	schubfach	json-canon	schubfach*	1.69×	[1.679, 1.700]	0.0002	0.0003	-71.62	
go:schubfach-vs-bd	api	verify	canonical/mixed-prod	schubfach	json-canon	schubfach*	1.32×	[1.314, 1.325]	0.0002	0.0003	-53.90	
go:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach	json-canon	schubfach*	1.40×	[1.394, 1.406]	0.0002	0.0003	-72.78	
rs:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.93×	[1.873, 1.999]	0.0002	0.0003	-16.40	
schubfach:go-vs-rs	api	verify	canonical/nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.62×	[2.551, 2.691]	0.0002	0.0003	47.98	
bd:go-vs-rs	api	verify	canonical/nested-mixed	json-canon	json-canon-rs	json-canon-rs*	1.89×	[1.848, 1.927]	0.0002	0.0003	35.89	
go:schubfach-vs-bd	api	verify	canonical/number-heavy	schubfach	json-canon	schubfach*	1.55×	[1.541, 1.562]	0.0002	0.0003	-49.40	
go:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach	json-canon	schubfach*	1.47×	[1.463, 1.486]	0.0002	0.0003	-37.54	
rs:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	9.09×	[8.972, 9.208]	0.0002	0.0003	-136.66	
schubfach:go-vs-rs	api	verify	canonical/numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	13.31×	[13.134, 13.465]	0.0002	0.0003	194.58	
bd:go-vs-rs	api	verify	canonical/numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.16×	[2.141, 2.177]	0.0002	0.0003	63.17	
go:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	schubfach	json-canon	json-canon	1.00×	[1.001, 1.009]	0.0462	0.0554	0.97	
rs:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	schubfach-rs	json-canon-rs	schubfach-rs	1.01×	[0.963, 1.040]	0.7556	0.7842	-0.15	
schubfach:go-vs-rs	api	verify	canonical/rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.18×	[2.111, 2.225]	0.0002	0.0003	36.57	
bd:go-vs-rs	api	verify	canonical/rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.15×	[2.101, 2.218]	0.0002	0.0003	34.01	
go:schubfach-vs-bd	api	verify	canonical/small	schubfach	json-canon	schubfach*	1.35×	[1.348, 1.360]	0.0002	0.0003	-55.77	
rs:schubfach-vs-bd	api	verify	canonical/small	schubfach-rs	json-canon-rs	schubfach-rs*	1.68×	[1.604, 1.760]	0.0002	0.0003	-9.56	
schubfach:go-vs-rs	api	verify	canonical/small	schubfach	schubfach-rs	schubfach-rs*	2.03×	[1.945, 2.103]	0.0002	0.0003	22.07	
bd:go-vs-rs	api	verify	canonical/small	json-canon	json-canon-rs	json-canon-rs*	1.63×	[1.572, 1.667]	0.0002	0.0003	19.24	
go:schubfach-vs-bd	api	verify	canonical/surrogate-pair	schubfach	json-canon	json-canon*	1.01×	[1.005, 1.012]	0.0006	0.0008	1.89	
rs:schubfach-vs-bd	api	verify	canonical/surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs	1.05×	[0.976, 1.166]	0.3519	0.3813	-0.46	
schubfach:go-vs-rs	api	verify	canonical/surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.33×	[2.234, 2.414]	0.0002	0.0003	28.19	
bd:go-vs-rs	api	verify	canonical/surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.20×	[1.991, 2.344]	0.0002	0.0003	12.38	
go:schubfach-vs-bd	api	verify	canonical/unicode	schubfach	json-canon	json-canon	1.00×	[0.997, 1.006]	0.6795	0.7111	0.19	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
rs:schubfach-vs-bd	api	verify	canonical/unicode	schubfach-rs	json-canon-rs	json-canon-rs*	1.04×	[1.024, 1.059]	0.0008	0.0011	1.95	
schubfach:go-vs-rs	api	verify	canonical/unicode	schubfach	schubfach-rs	schubfach*	1.10×	[1.083, 1.115]	0.0004	0.0005	-5.25	
bd:go-vs-rs	api	verify	canonical/unicode	json-canon	json-canon-rs	json-canon*	1.05×	[1.045, 1.066]	0.0002	0.0003	-4.27	
go:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach	json-canon	schubfach*	1.70×	[1.695, 1.709]	0.0004	0.0005	-101.83	
rs:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	2.69×	[2.510, 2.823]	0.0002	0.0003	-19.35	
schubfach:go-vs-rs	api	verify	canonical/verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.57×	[2.400, 2.689]	0.0002	0.0003	23.37	
bd:go-vs-rs	api	verify	canonical/verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	1.62×	[1.595, 1.652]	0.0002	0.0003	28.58	
go:schubfach-vs-bd	api	verify	noncanonical/control-escapes	schubfach	json-canon	json-canon*	1.05×	[1.042, 1.054]	0.0004	0.0005	6.85	
go:schubfach-vs-bd	api	verify	noncanonical/escaped-key-order	schubfach	json-canon	schubfach*	1.60×	[1.596, 1.613]	0.0002	0.0003	-65.87	
go:schubfach-vs-bd	api	verify	noncanonical/large	schubfach	json-canon	schubfach*	1.72×	[1.711, 1.730]	0.0002	0.0003	-75.97	
go:schubfach-vs-bd	api	verify	noncanonical/medium	schubfach	json-canon	schubfach*	1.67×	[1.664, 1.680]	0.0002	0.0003	-80.86	
go:schubfach-vs-bd	api	verify	noncanonical/mixed-prod	schubfach	json-canon	schubfach*	1.30×	[1.293, 1.306]	0.0002	0.0003	-41.27	
go:schubfach-vs-bd	api	verify	noncanonical/nested-mixed	schubfach	json-canon	schubfach*	1.39×	[1.388, 1.401]	0.0002	0.0003	-54.58	
go:schubfach-vs-bd	api	verify	noncanonical/number-heavy	schubfach	json-canon	schubfach*	1.54×	[1.526, 1.552]	0.0004	0.0005	-40.91	
go:schubfach-vs-bd	api	verify	noncanonical/numeric-boundary	schubfach	json-canon	schubfach*	1.47×	[1.461, 1.480]	0.0002	0.0003	-50.77	
go:schubfach-vs-bd	api	verify	noncanonical/rfc-key-sorting	schubfach	json-canon	json-canon*	1.02×	[1.016, 1.027]	0.0002	0.0003	3.26	
go:schubfach-vs-bd	api	verify	noncanonical/small	schubfach	json-canon	schubfach*	1.34×	[1.332, 1.345]	0.0002	0.0003	-48.33	
go:schubfach-vs-bd	api	verify	noncanonical/surrogate-pair	schubfach	json-canon	json-canon*	1.02×	[1.014, 1.023]	0.0002	0.0003	3.82	
go:schubfach-vs-bd	api	verify	noncanonical/unicode	schubfach	json-canon	json-canon*	1.01×	[1.002, 1.015]	0.0160	0.0202	1.21	
go:schubfach-vs-bd	api	verify	noncanonical/verify-whitespace	schubfach	json-canon	schubfach*	1.70×	[1.694, 1.710]	0.0002	0.0003	-106.42	

*Significant after Benjamini-Hochberg FDR correction at $q = 0.05$.
d = Cohen's d: $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster. Track: e2e = end-to-end CLI.
Noise = CV × mean of faster impl. MDE = minimum detectable effect at 80% power (z-approx).
Rust allocs/op and B/op report 0 (not measured); see §8.